

Day 13 Hands-on Assignment

Spring Cloud Service Discovery and Configuration: An E-Commerce Use Case

John Michael Bilbao
Techstart
August 28, 2026

The application

Bilbao Bazaar is a small e-commerce back end split into two business services. The Product Service owns the catalogue: eight products, their prices and their stock. The Order Service places orders, and to price one it has to ask the Product Service what the item costs.

That single cross-service call is where the whole assignment lives. The Order Service has to find the Product Service without being told where it is, which is service discovery, and both services have to agree on the tax rate and the currency without either of them declaring it, which is configuration management. Two more applications exist to make that possible: a Eureka server that holds the registry, and a Config Server that holds the configuration.

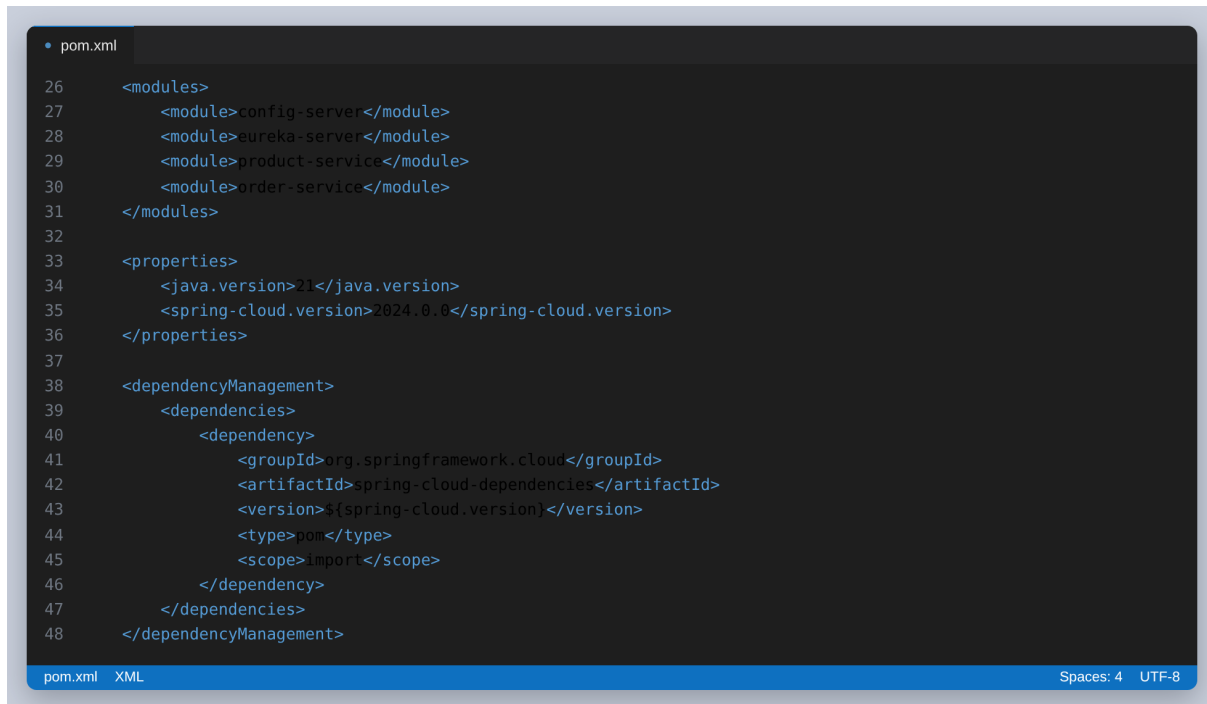
Application	Port	What it does
config-server	8888	Serves the YAML in config-repo/ over HTTP
eureka-server	8761	The service registry, and the dashboard
product-service	9081	The catalogue. Registers with Eureka, configured by the Config Server
order-service	9082	Places orders. Finds the Product Service through Eureka

Everything below ran on Java 21.0.2 with Spring Boot 3.4.1 and Spring Cloud 2024.0.0. The Eureka dashboard, the Config Server and both service APIs were photographed in a real browser at the URLs shown in each address bar; the terminal figures are captured command output. Nothing is retyped, so no figure can show something the applications did not do.

The one design decision worth stating up front: no host and no port for the Product Service appears anywhere in the Order Service. It calls `http://product-service/products/{id}` and lets the registry resolve the name. The proof that this is real rather than incidental comes at the end, where the Product Service is restarted on a different port (9091 instead of 9081) and the Order Service keeps working without a restart or a configuration change.

Step 1: setting up the environment

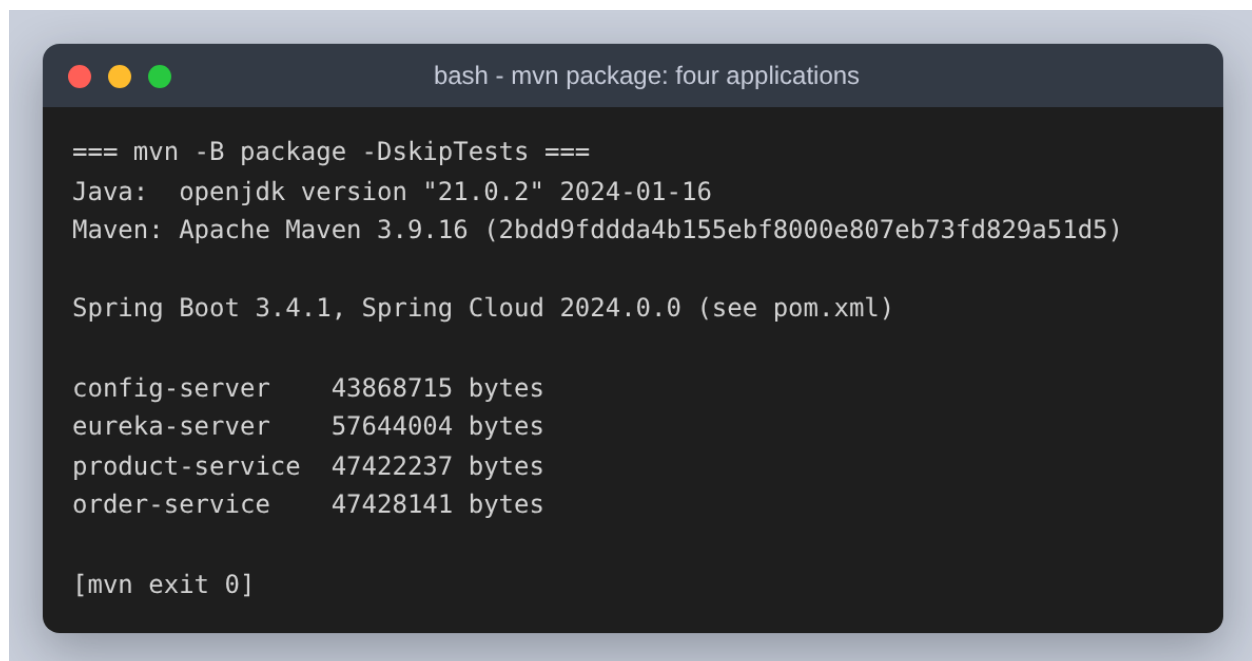
The four applications are Maven modules under one aggregator pom. Declaring the Spring Cloud BOM once there is what keeps the Eureka and Config dependencies in the modules version-free, and stops the two from drifting apart.



```
26     <modules>
27         <module>config-server</module>
28         <module>eureka-server</module>
29         <module>product-service</module>
30         <module>order-service</module>
31     </modules>
32
33     <properties>
34         <java.version>21</java.version>
35         <spring-cloud.version>2024.0.0</spring-cloud.version>
36     </properties>
37
38     <dependencyManagement>
39         <dependencies>
40             <dependency>
41                 <groupId>org.springframework.cloud</groupId>
42                 <artifactId>spring-cloud-dependencies</artifactId>
43                 <version>${spring-cloud.version}</version>
44                 <type>pom</type>
45                 <scope>import</scope>
46             </dependency>
47         </dependencies>
48     </dependencyManagement>
```

Figure 1 shows a code editor window with a tab labeled 'pom.xml'. The editor displays the XML content of the aggregator pom.xml file, including the module list and the Spring Cloud BOM dependency management. The status bar at the bottom indicates 'pom.xml XML' and 'Spaces: 4 UTF-8'.

Figure 1: The aggregator pom: the module list, and the Spring Cloud BOM imported once for all four.



```
bash - mvn package: four applications

=== mvn -B package -DskipTests ===
Java:   openjdk version "21.0.2" 2024-01-16
Maven:  Apache Maven 3.9.16 (2bdd9fddda4b155ebf8000e807eb73fd829a51d5)

Spring Boot 3.4.1, Spring Cloud 2024.0.0 (see pom.xml)

config-server      43868715 bytes
eureka-server      57644004 bytes
product-service    47422237 bytes
order-service      47428141 bytes

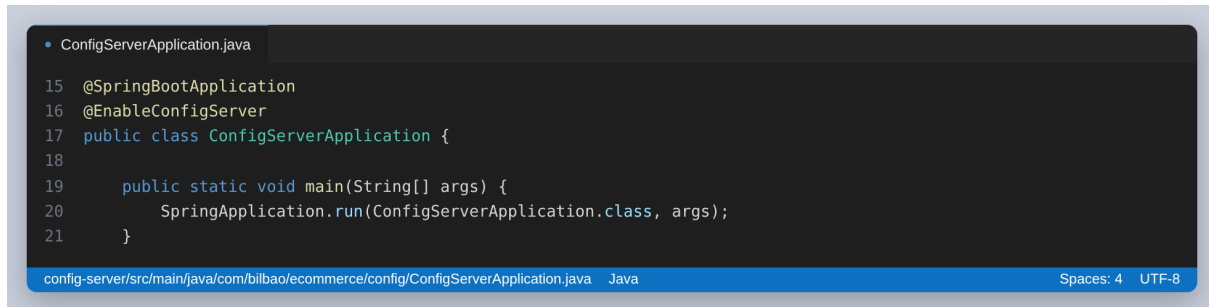
[mvn exit 0]
```

Figure 2 shows a terminal window with the title 'bash - mvn package: four applications'. It displays the output of the command 'mvn -B package -DskipTests', showing the Java version, Maven version, Spring Boot version, and the sizes of the four generated JAR files.

Figure 2: One mvn package builds all four executable jars.

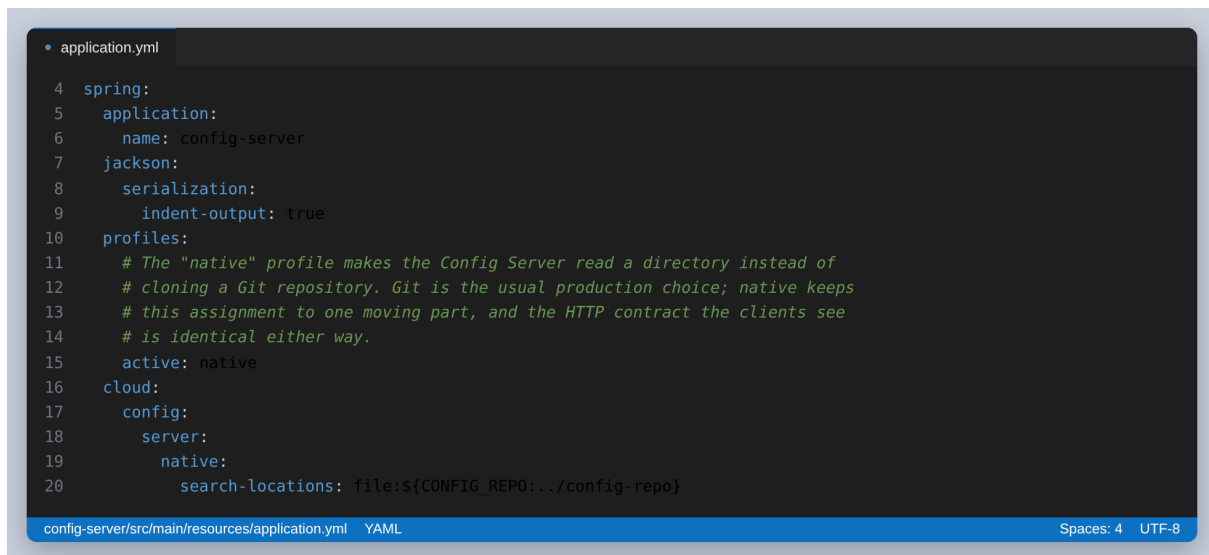
Step 3 comes first: the Config Server

The assignment lists configuration third, but it has to be built second, because the two business services ask the Config Server for their configuration during startup rather than after it. With no Config Server running there is nothing for them to be configured from, so it is the first thing that has to exist after the build.



```
• ConfigServerApplication.java
15 @SpringBootApplication
16 @EnableConfigServer
17 public class ConfigServerApplication {
18
19     public static void main(String[] args) {
20         SpringApplication.run(ConfigServerApplication.class, args);
21     }
}
config-server/src/main/java/com/bilbao/ecommerce/config/ConfigServerApplication.java  Java  Spaces: 4  UTF-8
```

Figure 3: The entire Config Server. One annotation.



```
• application.yml
4 spring:
5   application:
6     name: config-server
7   jackson:
8     serialization:
9       indent-output: true
10  profiles:
11    # The "native" profile makes the Config Server read a directory instead of
12    # cloning a Git repository. Git is the usual production choice; native keeps
13    # this assignment to one moving part, and the HTTP contract the clients see
14    # is identical either way.
15    active: native
16  cloud:
17    config:
18      server:
19        native:
20          search-locations: file:${CONFIG_REPO:../config-repo}
config-server/src/main/resources/application.yml  YAML  Spaces: 4  UTF-8
```

Figure 4: The native profile points the server at a directory instead of a Git repository. Git is the usual production choice; the HTTP contract the clients see is identical either way, and this keeps the assignment to one moving part.

The configuration itself is split in two, and the split is the argument for having a Config Server at all. Anything both services need lives in application.yml. Anything only one service needs lives in a file named after that service.

```
• application.yml
4  spring:
5    jackson:
6      serialization:
7        # Indented JSON, so that a response read in a browser is legible without
8        # a formatting extension. This is served from here rather than set in each
9        # service, which is itself a small demonstration of the point.
10       indent-output: true
11
12  store:
13    name: Bilbao Bazaar
14    currency: PHP
15    tax-rate: 0.12
16    support-email: support@bilbao-bazaar.example
17
18  management:
19    endpoints:
20      web:
21        exposure:
22          # /actuator/refresh is what makes a configuration change reachable
23          # without a restart. It is the endpoint Step 3 posts to.
24          include: health,info,refresh,env,configprops
25    endpoint:
26      health:
27        show-details: always
config-repo/application.yml  YAML  Spaces: 4  UTF-8
```

Figure 5: config-repo/application.yml: the store name, currency and tax rate, defined once for every service that asks.

```
• product-service.yml
1  # Configuration served to product-service. The service itself ships with none
2  # of this; it arrives over HTTP from the Config Server at startup.
3  catalog:
4    featured-message: Welcome to Bilbao Bazaar
5    page-size: 20
6    low-stock-threshold: 5
config-repo/product-service.yml  YAML  Spaces: 4  UTF-8
```

Figure 6: config-repo/product-service.yml. The file name is not arbitrary: it has to match the service's `spring.application.name`, which is how the server knows which file to serve to whom.

The server exposes it at `/[{application}]/[{profile}]`, and merges the two files for the caller. Asking for the shared file directly shows what every service receives:

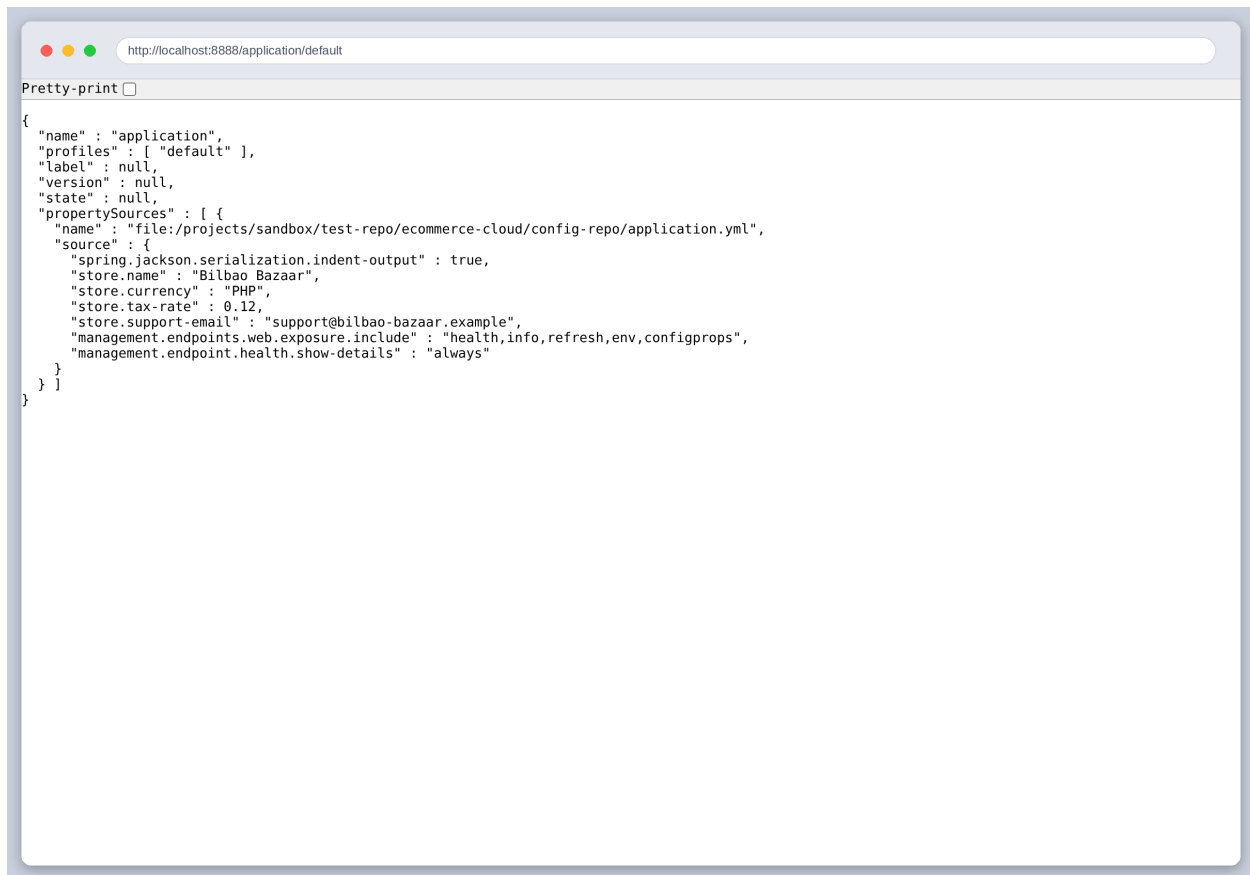


Figure 7: The Config Server serving the shared configuration, in a browser at localhost:8888/application/default.

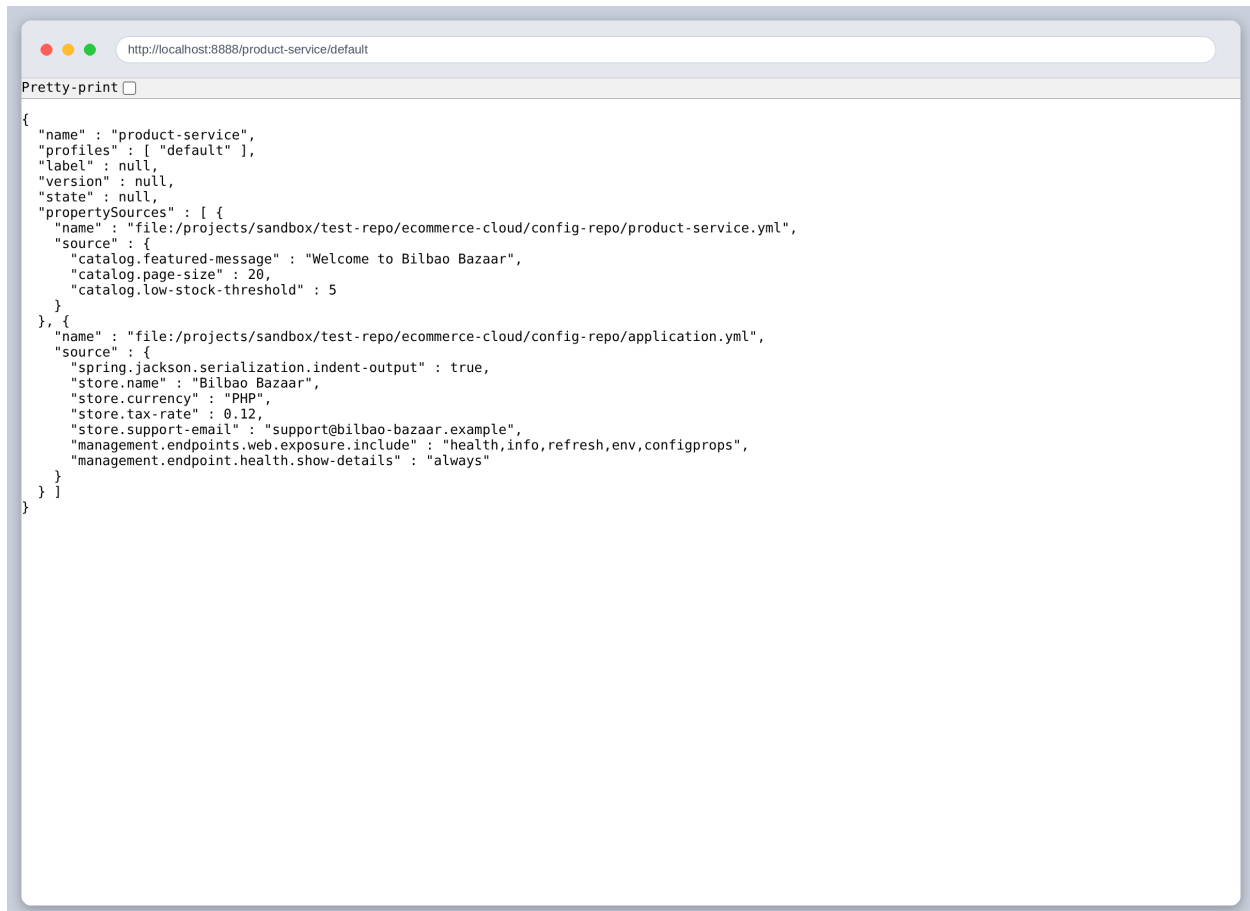
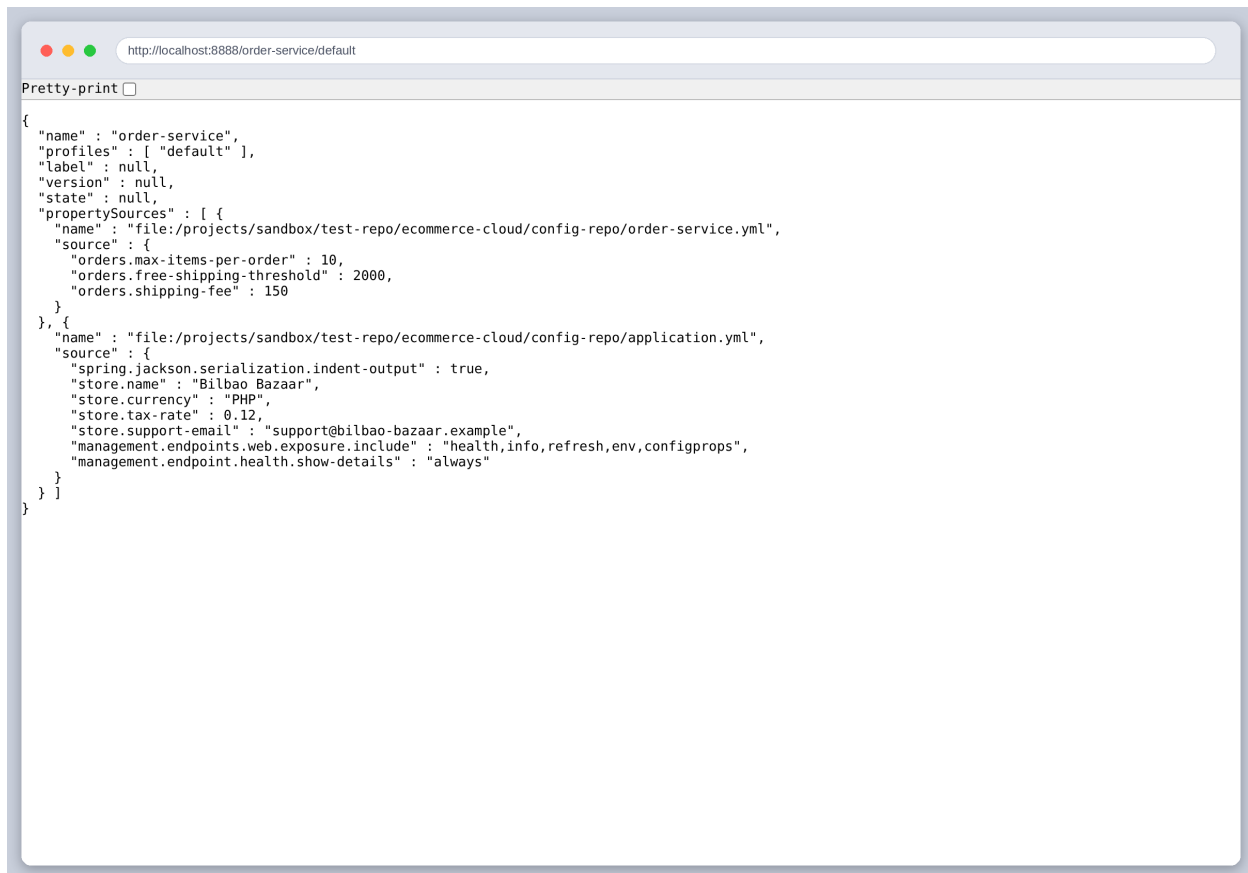
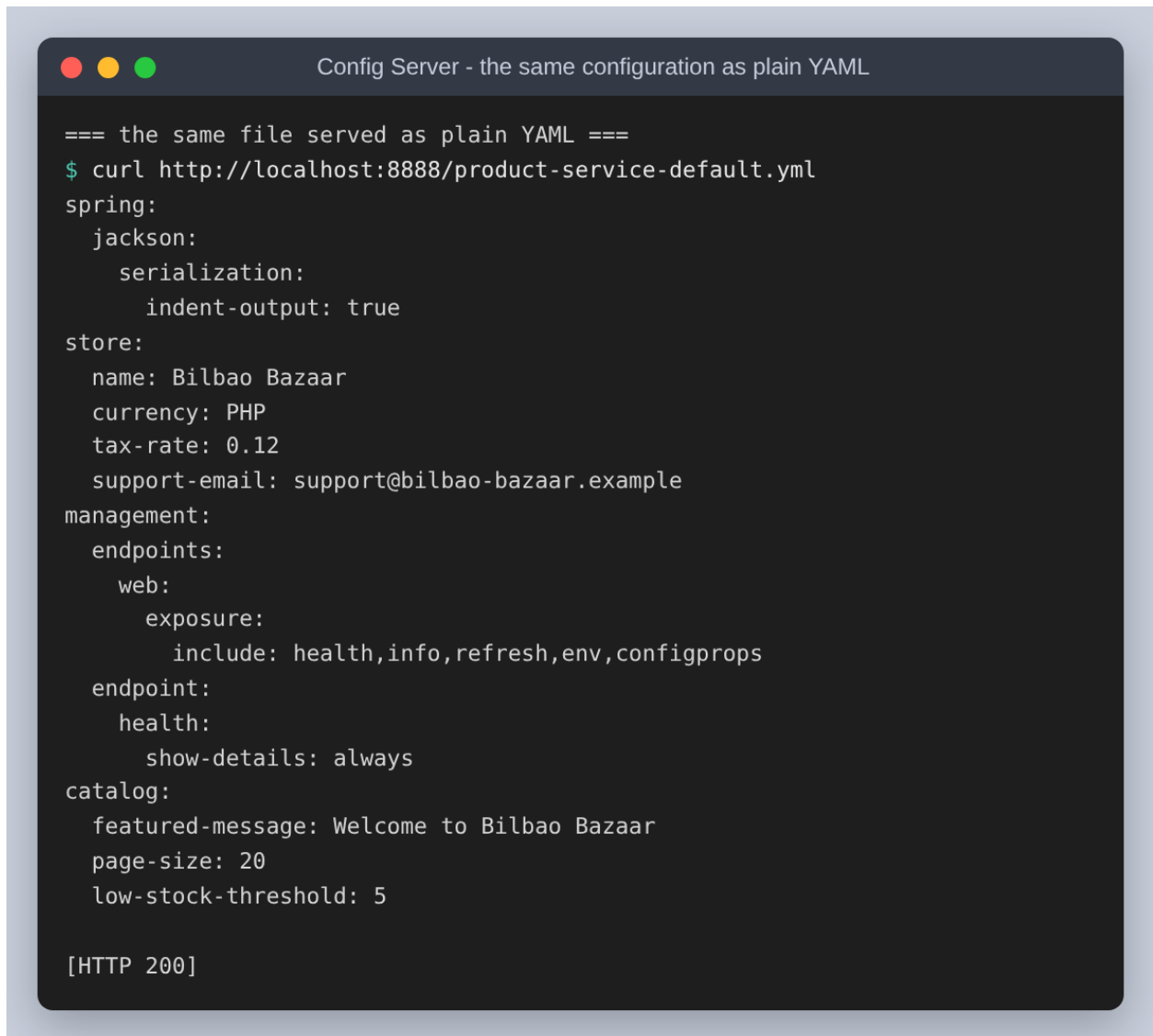


Figure 8: The same endpoint for product-service. Two property sources come back: the service-specific file first, then the shared one. That order is the precedence rule, and it is why a service can override a shared default.



```
{
  "name" : "order-service",
  "profiles" : [ "default" ],
  "label" : null,
  "version" : null,
  "state" : null,
  "propertySources" : [ {
    "name" : "file:/projects/sandbox/test-repo/ecommerce-cloud/config-repo/order-service.yml",
    "source" : {
      "orders.max-items-per-order" : 10,
      "orders.free-shipping-threshold" : 2000,
      "orders.shipping-fee" : 150
    }
  } ],
  {
    "name" : "file:/projects/sandbox/test-repo/ecommerce-cloud/config-repo/application.yml",
    "source" : {
      "spring.jackson.serialization.indent-output" : true,
      "store.name" : "Bilbao Bazaar",
      "store.currency" : "PHP",
      "store.tax-rate" : 0.12,
      "store.support-email" : "support@bilbao-bazaar.example",
      "management.endpoints.web.exposure.include" : "health,info,refresh,env,configprops",
      "management.endpoint.health.show-details" : "always"
    }
  }
}
```

Figure 9: And for order-service, which gets its own orders.* values plus the same shared store.* block.



```
Config Server - the same configuration as plain YAML

=== the same file served as plain YAML ===
$ curl http://localhost:8888/product-service-default.yml
spring:
  jackson:
    serialization:
      indent-output: true
store:
  name: Bilbao Bazaar
  currency: PHP
  tax-rate: 0.12
  support-email: support@bilbao-bazaar.example
management:
  endpoints:
    web:
      exposure:
        include: health,info,refresh,env,configprops
  endpoint:
    health:
      show-details: always
catalog:
  featured-message: Welcome to Bilbao Bazaar
  page-size: 20
  low-stock-threshold: 5

[HTTP 200]
```

Figure 10: The same configuration served as plain YAML rather than as the server's JSON envelope, which is the form that is easiest to check by eye.

Step 2: service discovery with Eureka

The Eureka server is the registry: services report in when they start, renew a lease while they are alive, and are removed when they stop or stop renewing. It is as small as the Config Server.

```
• EurekaServerApplication.java
15 @SpringBootApplication
16 @EnableEurekaServer
17 public class EurekaServerApplication {
18
19     public static void main(String[] args) {
20         SpringApplication.run(EurekaServerApplication.class, args);
21     }
22 }
```

eureka-server/src/main/java/com/bilbao/ecommerce/eureka/EurekaServerApplication.java Java Spaces: 4 UTF-8

Figure 11: The Eureka server. It registers nothing itself, which is why both client flags are false in its configuration.

```
• application.yml
8 eureka:
9   client:
10     # This IS the registry, so it neither registers with itself nor fetches a
11     # copy of its own registry.
12     register-with-eureka: false
13     fetch-registry: false
14   server:
15     # Self preservation stops Eureka evicting instances when it thinks it is
16     # seeing a network partition rather than a genuine shutdown. That is the
17     # right default in production and the wrong one here: Step 2 has to show a
18     # service actually disappearing from the registry, and with this on the
19     # dead instance would linger. eviction-interval makes the sweep prompt.
20     enable-self-preservation: false
21     eviction-interval-timer-in-ms: 3000
22     response-cache-update-interval-ms: 1000
```

eureka-server/src/main/resources/application.yml YAML Spaces: 4 UTF-8

Figure 12: Two settings here are deliberate departures from the defaults, and the comment says why: self preservation would stop Eureka evicting a stopped instance, and this assignment has to show one actually disappearing.

Eureka REST API - the registry before anything registers

```
=== the registry, empty apart from itself ===
$ curl -H Accept: application/json http://localhost:8761/eureka/apps
{"applications":{"versions__delta":"1","apps__hashcode":"","application":[]}}
[HTTP 200]
```

Figure 13: The registry immediately after the server starts: empty.

A service becomes a client by adding the Eureka client starter and pointing it at the registry. Its `spring.application.name` does double duty: Eureka registers the instance under it, and the Config Server uses it to choose a file.

```
• application.yml
4  spring:
5    application:
6      # This name does two things: Eureka registers the instance under it, and the
7      # Config Server uses it to pick which YAML file to serve. That is why
8      # config-repo/product-service.yml is named the way it is.
9      name: product-service
10   config:
11     # Ask the Config Server for configuration before the application context is
12     # built. optional: means the service still starts if the Config Server is
13     # down, which is how Step 3 can show the difference between configured and
14     # unconfigured rather than just failing to boot.
15     import: optional:configserver:${CONFIG_SERVER_URI:http://localhost:8888}
16
17   eureka:
18     client:
19       service-url:
20         defaultZone: ${EUREKA_URI:http://localhost:8761/eureka}
21       # Short intervals so registration and deregistration are quick enough to
22       # screenshot. Production defaults are 30s each.
23       registry-fetch-interval-seconds: 5
24     instance:
25       prefer-ip-address: true
26       lease-renewal-interval-in-seconds: 5
27       lease-expiration-duration-in-seconds: 10
```

product-service/src/main/resources/application.yml YAML Spaces: 4 UTF-8

Figure 14: All the Product Service knows: its own name, where the Config Server is, and where Eureka is. The lease intervals are shortened from the 30-second defaults so registration and eviction are quick enough to observe.

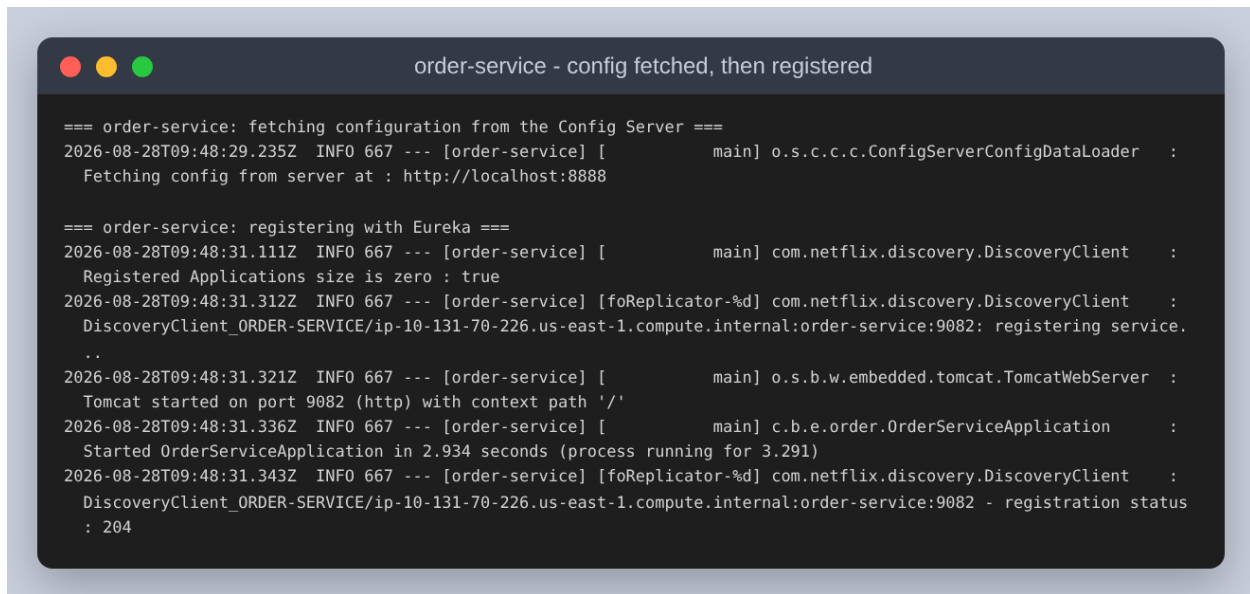
Both services then start, and the startup logs show the two mechanisms in sequence: fetch configuration, then register.

```
product-service - config fetched, then registered

=== product-service: fetching configuration from the Config Server ===
2026-08-28T09:48:25.090Z INFO 604 --- [product-service] [      main] o.s.c.c.c.ConfigServerConfigDataLoader
: Fetching config from server at : http://localhost:8888

=== product-service: registering with Eureka ===
2026-08-28T09:48:27.103Z INFO 604 --- [product-service] [      main] com.netflix.discovery.DiscoveryClient
: Registered Applications size is zero : true
2026-08-28T09:48:27.290Z INFO 604 --- [product-service] [foReplicator-%d] com.netflix.discovery.DiscoveryClient
: DiscoveryClient_PRODUCT-SERVICE/ip-10-131-70-226.us-east-1.compute.internal:product-service:9081: registering se
rvice...
2026-08-28T09:48:27.300Z INFO 604 --- [product-service] [      main] o.s.b.w.embedded.tomcat.TomcatWebServer
: Tomcat started on port 9081 (http) with context path '/'
2026-08-28T09:48:27.320Z INFO 604 --- [product-service] [      main] c.b.e.product.ProductServiceApplication
: Started ProductServiceApplication in 2.998 seconds (process running for 3.389)
2026-08-28T09:48:27.423Z INFO 604 --- [product-service] [foReplicator-%d] com.netflix.discovery.DiscoveryClient
: DiscoveryClient_PRODUCT-SERVICE/ip-10-131-70-226.us-east-1.compute.internal:product-service:9081 - registration
status: 204
2026-08-28T09:48:32.284Z INFO 604 --- [product-service] [reshExecutor-%d] com.netflix.discovery.DiscoveryClient
: Registered Applications size is zero : true
```

Figure 15: product-service: it fetches its configuration from localhost:8888, then registers with Eureka and Tomcat starts on 9081. Registration status 204 is Eureka acknowledging the lease.



```
order-service - config fetched, then registered

=== order-service: fetching configuration from the Config Server ===
2026-08-28T09:48:29.235Z INFO 667 --- [order-service] [main] o.s.c.c.c.ConfigServerConfigDataLoader :
    Fetching config from server at : http://localhost:8888

=== order-service: registering with Eureka ===
2026-08-28T09:48:31.111Z INFO 667 --- [order-service] [main] com.netflix.discovery.DiscoveryClient :
    Registered Applications size is zero : true
2026-08-28T09:48:31.312Z INFO 667 --- [order-service] [foReplicator-%d] com.netflix.discovery.DiscoveryClient :
    DiscoveryClient_ORDER-SERVICE/ip-10-131-70-226.us-east-1.compute.internal:order-service:9082: registering service.
..
2026-08-28T09:48:31.321Z INFO 667 --- [order-service] [main] o.s.b.w.embedded.tomcat.TomcatWebServer :
    Tomcat started on port 9082 (http) with context path '/'
2026-08-28T09:48:31.336Z INFO 667 --- [order-service] [main] c.b.e.order.OrderServiceApplication :
    Started OrderServiceApplication in 2.934 seconds (process running for 3.291)
2026-08-28T09:48:31.343Z INFO 667 --- [order-service] [foReplicator-%d] com.netflix.discovery.DiscoveryClient :
    DiscoveryClient_ORDER-SERVICE/ip-10-131-70-226.us-east-1.compute.internal:order-service:9082 - registration status
    : 204
```

Figure 16: order-service doing the same thing.



Figure 17: The Eureka dashboard with both services registered and UP. This is the screenshot the assignment asks for.

```
Eureka REST API - both services registered

=== both services registered ===
$ curl -H Accept: application/json http://localhost:8761/eureka/apps
{"applications":{"versions__delta":"1","apps__hashcode":"UP_2_","application":[{"name":"ORDER-SERVICE","instance":[{"instanceId":"ip-10-131-70-226.us-east-1.compute.internal:order-service:9082","hostName":"169.254.255.194","app":"ORDER-SERVICE","ipAddr":"169.254.255.194","status":"UP","overriddenStatus":"UNKNOWN","port":{"$":9082,"@enabled":"true"},"securePort":{"$":443,"@enabled":"false"},"countryId":1,"dataCenterInfo":{"@class":"com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo","name":"MyOwn"},"leaseInfo":{"renewalIntervalInSecs":5,"durationInSecs":10,"registrationTimestamp":1787910511854,"lastRenewalTimestamp":1787910511854,"evictionTimestamp":0,"serviceUpTimestamp":1787910511341},"metadata":{"management.port":"9082"},"homePageUrl":"http://169.254.255.194:9082/","statusPageUrl":"http://169.254.255.194:9082/actuator/info","healthCheckUrl":"http://169.254.255.194:9082/actuator/health","vipAddress":"order-service","secureVipAddress":"order-service","isCoordinatingDiscoveryServer":"false","lastUpdatedTimestamp":1787910511854,"lastDirtyTimestamp":1787910511310,"actionType":"ADDED"}]},{"name":"PRODUCT-SERVICE","instance":[{"instanceId":"ip-10-131-70-226.us-east-1.compute.internal:product-service:9081","hostName":"169.254.255.194","app":"PRODUCT-SERVICE","ipAddr":"169.254.255.194","status":"UP","overriddenStatus":"UNKNOWN","port":{"$":9081,"@enabled":"true"},"securePort":{"$":443,"@enabled":"false"},"countryId":1,"dataCenterInfo":{"@class":"com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo","name":"MyOwn"},"leaseInfo":{"renewalIntervalInSecs":5,"durationInSecs":10,"registrationTimestamp":1787910508043,"lastRenewalTimestamp":1787910508043,"evictionTimestamp":0,"serviceUpTimestamp":1787910507405},"metadata":{"management.port":"9081"},"homePageUrl":"http://169.254.255.194:9081/","statusPageUrl":"http://169.254.255.194:9081/actuator/info","healthCheckUrl":"http://169.254.255.194:9081/actuator/health","vipAddress":"product-service","secureVipAddress":"product-service","isCoordinatingDiscoveryServer":"false","lastUpdatedTimestamp":1787910508043,"lastDirtyTimestamp":1787910507288,"actionType":"ADDED"}]}}]}
[HTTP 200]
```

Figure 18: The same registry over Eureka's REST API rather than the dashboard, which is what a client actually consumes.

```
Eureka REST API - the product-service lease in full

=== the product-service lease in detail ===
$ curl -H Accept: application/json http://localhost:8761/eureka/apps/PRODUCT-SERVICE
{"application":{"name":"PRODUCT-SERVICE","instance":[{"instanceId":"ip-10-131-70-226.us-east-1.compute.internal:product-service:9081","hostName":"169.254.255.194","app":"PRODUCT-SERVICE","ipAddr":"169.254.255.194","status":"UP","overriddenStatus":"UNKNOWN","port":{"$":9081,"@enabled":"true"},"securePort":{"$":443,"@enabled":"false"},"countryId":1,"dataCenterInfo":{"@class":"com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo","name":"MyOwn"},"leaseInfo":{"renewalIntervalInSecs":5,"durationInSecs":10,"registrationTimestamp":1787910508043,"lastRenewalTimestamp":1787910532827,"evictionTimestamp":0,"serviceUpTimestamp":1787910507405},"metadata":{"management.port":"9081"},"homePageUrl":"http://169.254.255.194:9081/","statusPageUrl":"http://169.254.255.194:9081/actuator/info","healthCheckUrl":"http://169.254.255.194:9081/actuator/health","vipAddress":"product-service","secureVipAddress":"product-service","isCoordinatingDiscoveryServer":"false","lastUpdatedTimestamp":1787910508043,"lastDirtyTimestamp":1787910507288,"actionType":"ADDED"}]}}]}
[HTTP 200]
```

Figure 19: One lease in full. The interesting fields are `vipAddress`, the logical name callers use, and `leaseInfo`, which carries the renewal interval and the duration after which the lease expires.

Using the registry, rather than just filling it

Registering is the easy half. The Order Service has to resolve the name to an address, and that is one bean.

```

• OrderServiceApplication.java

21  /**
22   * The one bean that makes service discovery useful rather than merely
23   * present.
24   *
25   * <p>{@code @LoadBalanced} wires a Spring Cloud LoadBalancer interceptor
26   * into this RestTemplate. Given the URL {@code http://product-service/...},
27   * the interceptor treats the host as a name to look up in Eureka, picks one
28   * of the registered instances, and rewrites the URL to that instance's real
29   * host and port before the request is sent.
30   *
31   * <p>Without the annotation the same call fails with UnknownHostException,
32   * because "product-service" is not a DNS name. That failure is captured in
33   * the write-up, because it is the clearest demonstration that the lookup is
34   * genuinely happening here and not somewhere else.
35   */
36  @Bean
37  @LoadBalanced
38  public RestTemplate loadBalancedRestTemplate() {

order-service/src/main/java/com/bilbao/ecommerce/order/OrderServiceApplication.java  Java  Spaces: 4  UTF-8

```

Figure 20: The bean that makes discovery useful. `@LoadBalanced` adds an interceptor that treats the host in the URL as a name to look up in Eureka, picks a registered instance, and rewrites the URL before the request goes out.

```

• ProductClient.java

40  /** Fetch one product through Eureka. Empty if the product does not exist. */
41  public ProductView fetch(long productId) {
42      Map<?, ?> body = loadBalanced.getForObject(
43          "http://" + serviceName + "/products/{id}", Map.class, productId);
44      if (body == null) {
45          throw new ProductLookupException(
46              "product-service returned an empty body for product " + productId);
47      }
48      return new ProductView(
49          ((Number) body.get("id")).longValue(),
50          String.valueOf(body.get("name")),
51          new BigDecimal(String.valueOf(body.get("price"))),
52          ((Number) body.get("stock")).intValue());
53  }
54
55  /**
56   * The instances Eureka currently lists for the Product Service.
57   *
58   * <p>This is the registry as the Order Service sees it, which is not
59   * necessarily the same as the dashboard: the client caches the registry and
60   * refreshes it on an interval, so there is a window where the dashboard and
61   * this disagree. Step 2 shows that window.
62   */
63  public List<ServiceInstance> instances() {

order-service/src/main/java/com/bilbao/ecommerce/order/ProductClient.java  Java  Spaces: 4  UTF-8

```

Figure 21: The client. Every URL uses the logical name; no host or port appears anywhere in the Order Service.

It is worth proving that the lookup is really happening here, and not that something else is quietly resolving the name. The same call on a plain `RestTemplate`, with no interceptor, is exposed as its own endpoint:

```
• ProductClient.java

81     public String fetchWithoutDiscovery(long productId) {
82         try {
83             plain.getForObject("http://" + serviceName + "/products/{id}",
84                               String.class, productId);
85             return "unexpectedly succeeded";
86         } catch (RestClientException e) {
```

order-service/src/main/java/com/bilbao/e-commerce/order/ProductClient.java Java Spaces: 4 UTF-8

Figure 22: The deliberately broken call, kept as an endpoint so the failure can be captured rather than described.

```
order-service - the same URL without @LoadBalanced

=== the same URL without the load balancer ===
$ curl http://localhost:9082/orders/without-discovery/3
{
  "attempted" : "GET http://product-service/products/3 on a RestTemplate with no @LoadBalanced",
  "result" : "UnknownHostException: product-service"
}
[HTTP 200]
```

Figure 23: `UnknownHostException: product-service`. The name is not a DNS name and nothing on the machine can resolve it, so the working call must be going through the registry.

Step 4: the two services working together

With both mechanisms in place the application does something. The catalogue response is wrapped in values the Product Service did not declare: the store name and currency came from the shared file, the banner and page size from its own.

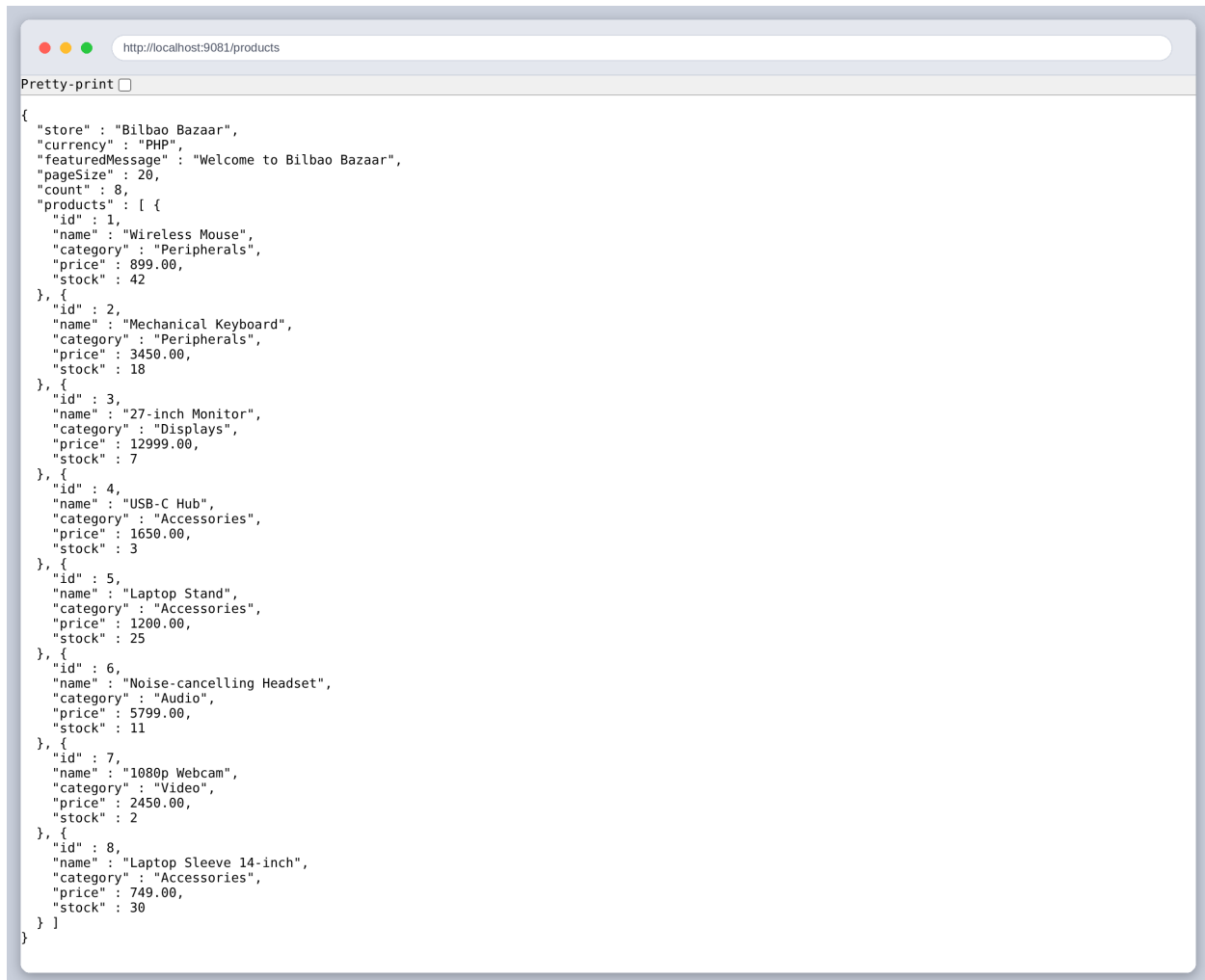
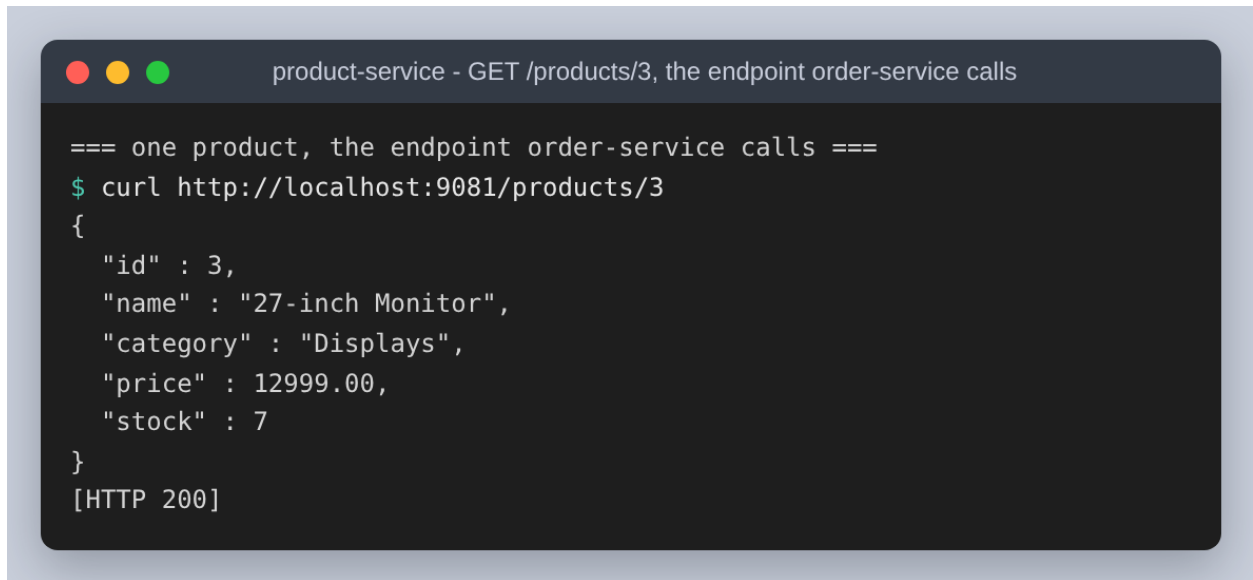


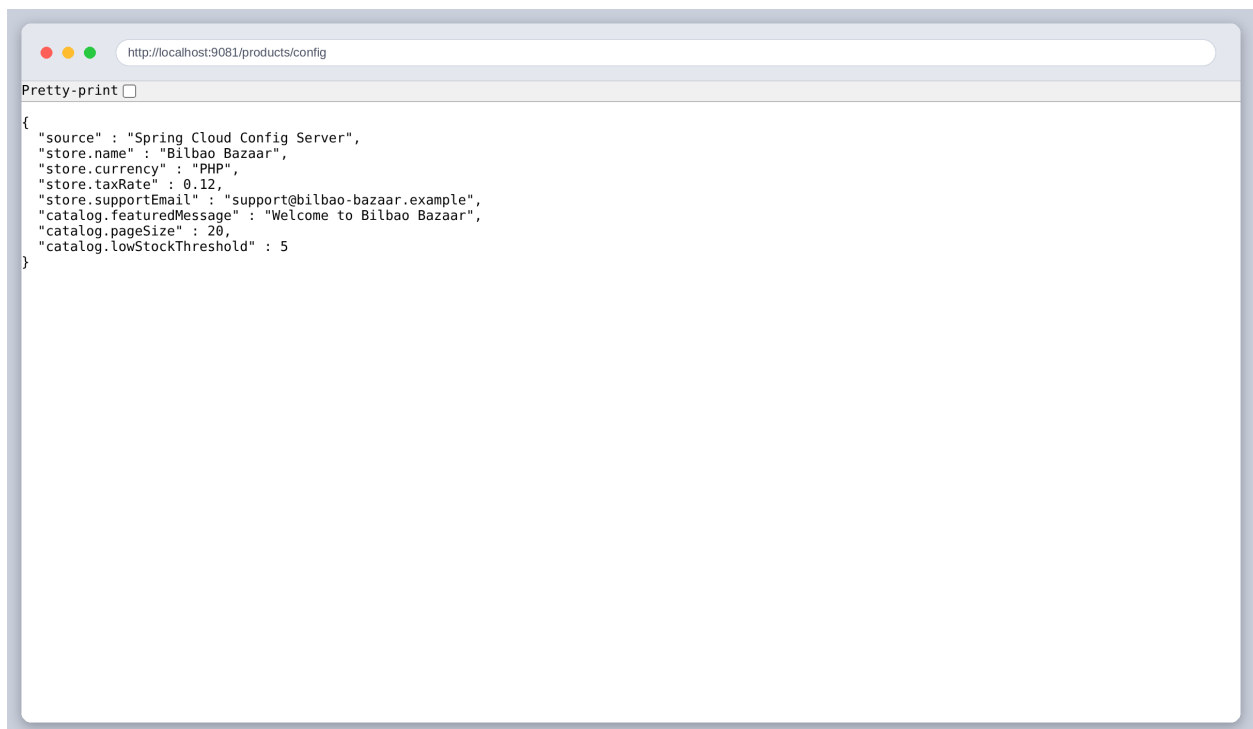
Figure 24: GET /products in a browser. featuredMessage and pageSize are configuration, not code.



```
product-service - GET /products/3, the endpoint order-service calls

=== one product, the endpoint order-service calls ===
$ curl http://localhost:9081/products/3
{
  "id" : 3,
  "name" : "27-inch Monitor",
  "category" : "Displays",
  "price" : 12999.00,
  "stock" : 7
}
[HTTP 200]
```

Figure 25: GET /products/3, the endpoint the Order Service calls through Eureka.



```
http://localhost:9081/products/config

Pretty-print ☐

{
  "source" : "Spring Cloud Config Server",
  "store.name" : "Bilbao Bazaar",
  "store.currency" : "PHP",
  "store.taxRate" : 0.12,
  "store.supportEmail" : "support@bilbao-bazaar.example",
  "catalog.featuredMessage" : "Welcome to Bilbao Bazaar",
  "catalog.pageSize" : 20,
  "catalog.lowStockThreshold" : 5
}
```

Figure 26: GET /products/config reports everything this service was told, which makes the effect of a configuration change visible in one place.

The Order Service exposes the same idea from the other side: one endpoint that answers whether discovery is working and whether configuration is working at the same time.

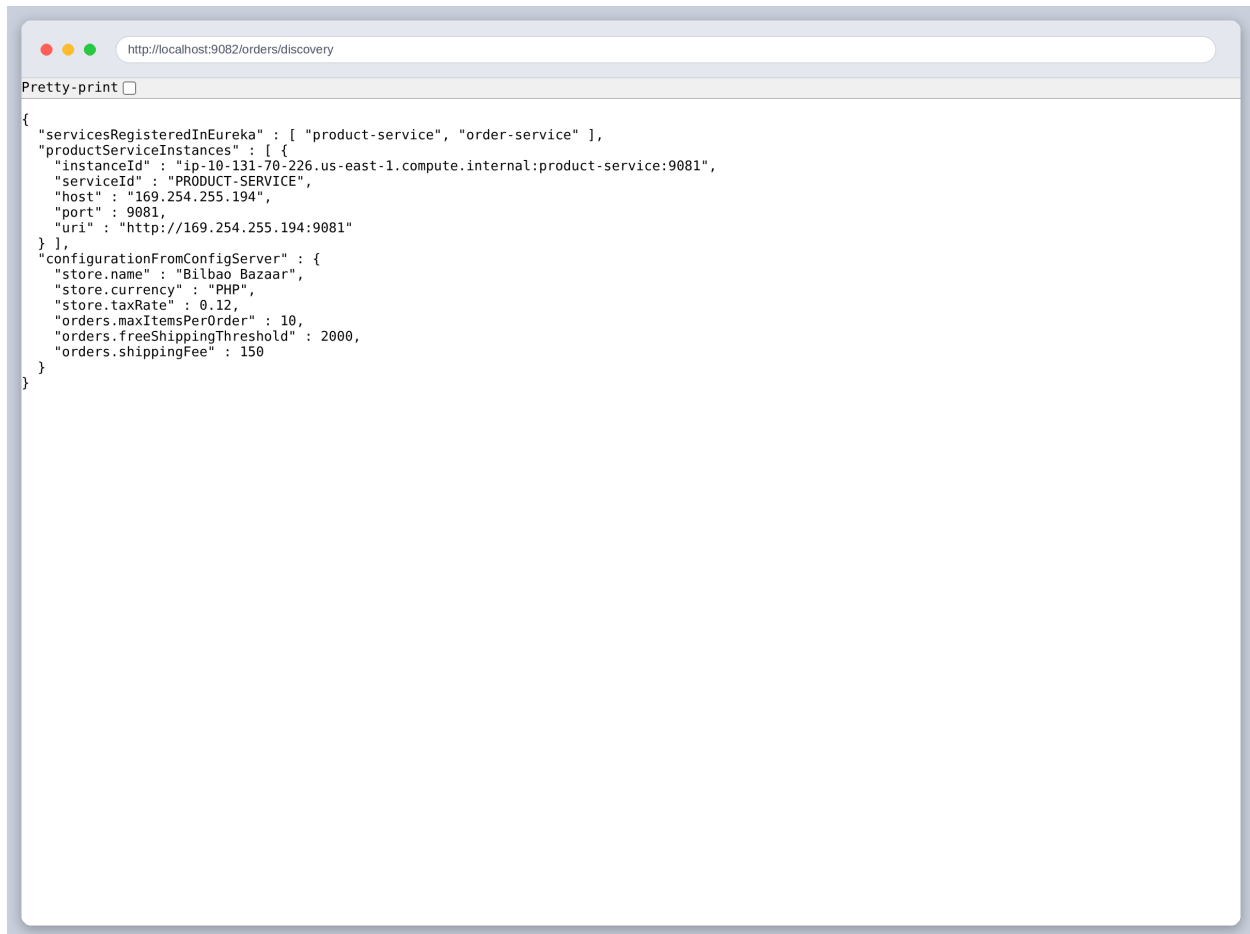


Figure 27: GET /orders/discovery: the service names Eureka knows, the concrete instances of product-service with their host and port, and the configuration this service received.

Placing an order uses both. The price is fetched from the Product Service found through Eureka; the tax rate, the shipping fee and the free-shipping threshold all come from the Config Server.

```
order-service - POST /orders, priced through Eureka

=== placing an order: 1 x 27-inch Monitor ===
$ curl -X POST -H Content-Type: application/json -d {"productId":3,"quantity":1} http://localhost:9082/orders
{
  "id" : "c307bce7",
  "productId" : 3,
  "productName" : "27-inch Monitor",
  "quantity" : 1,
  "unitPrice" : 12999.0,
  "subtotal" : 12999.00,
  "tax" : 1559.88,
  "shipping" : 0.00,
  "total" : 14558.88,
  "currency" : "PHP",
  "servedByProductInstance" : "169.254.255.194:9081",
  "placedAt" : "2026-08-28T09:48:44.312169720Z"
}
[HTTP 201]

=== placing a second order below the free shipping threshold ===
$ curl -X POST -H Content-Type: application/json -d {"productId":1,"quantity":1} http://localhost:9082/orders
{
  "id" : "28b970b0",
  "productId" : 1,
  "productName" : "Wireless Mouse",
  "quantity" : 1,
  "unitPrice" : 899.0,
  "subtotal" : 899.00,
  "tax" : 107.88,
  "shipping" : 150.00,
  "total" : 1156.88,
  "currency" : "PHP",
  "servedByProductInstance" : "169.254.255.194:9081",
  "placedAt" : "2026-08-28T09:48:44.330032951Z"
}
[HTTP 201]
```

Figure 28: Two orders. The monitor is over the 2,000 free-shipping threshold so shipping is zero; the mouse is under it and is charged 150. Both figures are configuration, and servedByProductInstance records which discovered instance answered.

```
order-service - a configured business rule refusing an order

=== a quantity above the configured maximum is refused ===
$ curl -X POST -H Content-Type: application/json -d {"productId":1,"quantity":25} http://localhost:9082/orders
{
  "error" : "quantity 25 exceeds the configured maximum of 10 items per order"
}
[HTTP 400]
```

Figure 29: A configured business rule refusing an order. The maximum is not a constant in the Order Service; it is a value the Config Server supplied, and the error message quotes it.

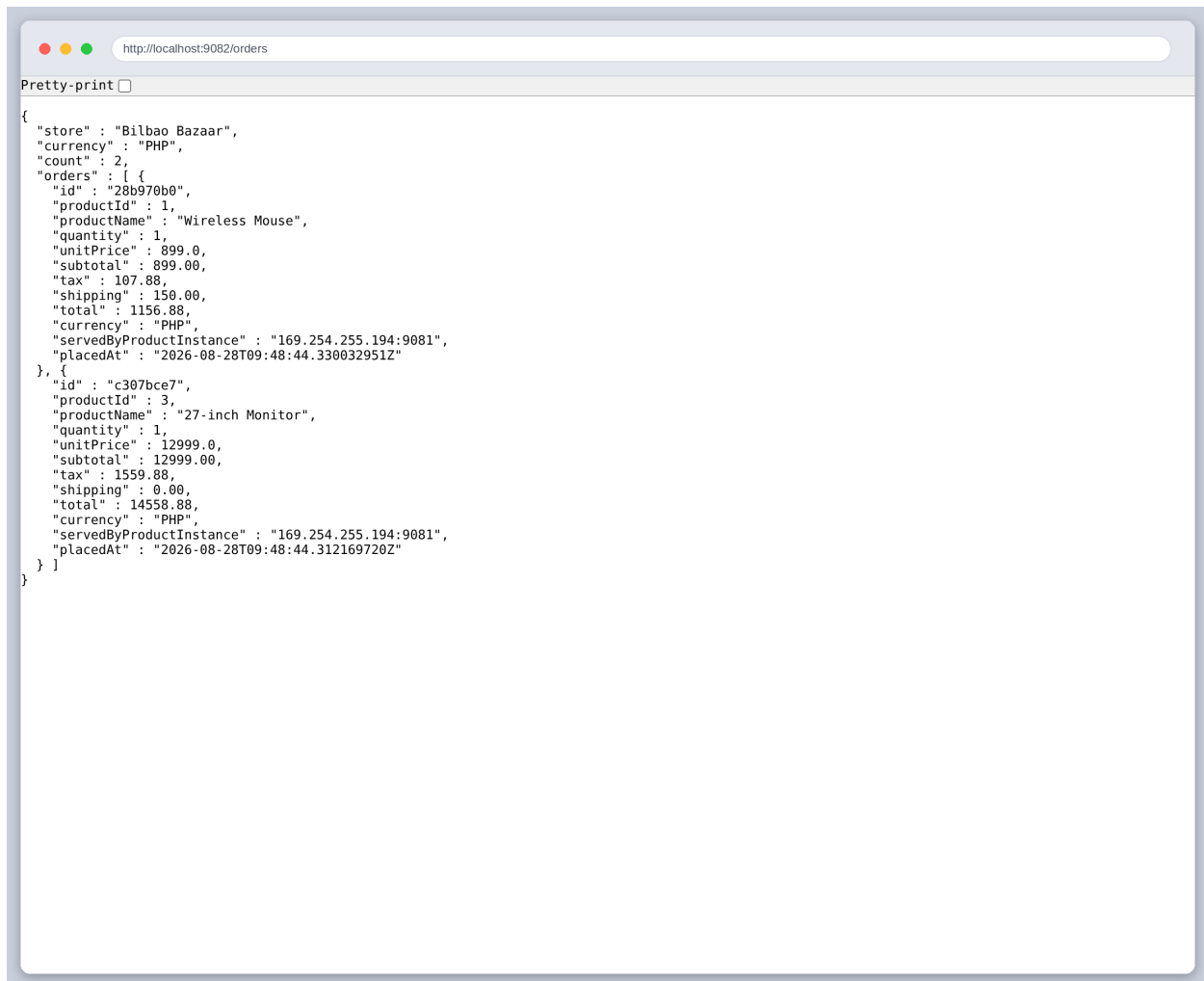
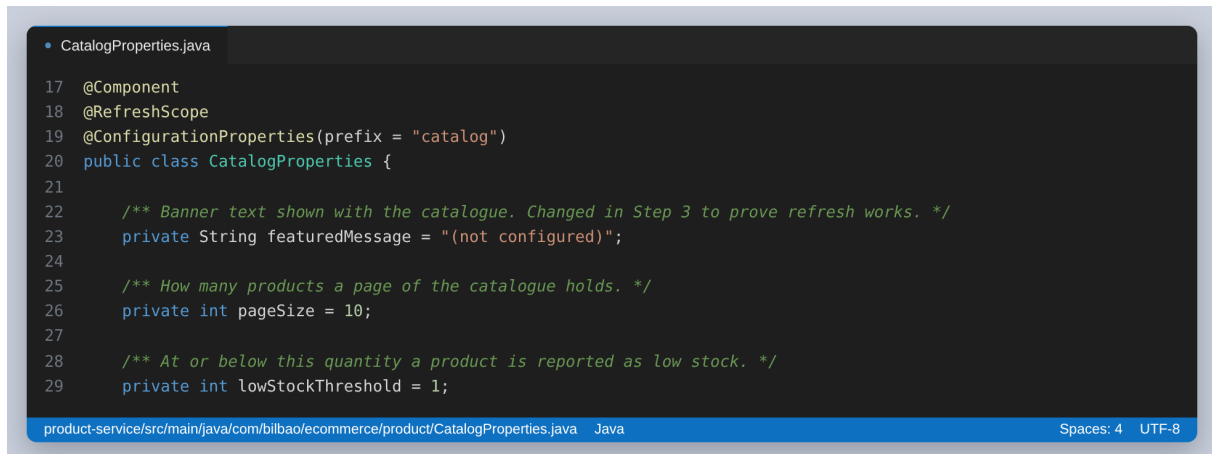


Figure 30: GET /orders in a browser, with both orders and the configured store name and currency.

One deliberate choice in the Order model: `productName` and `unitPrice` are copied from the Product Service when the order is placed, not looked up again on every read. An order is a record of what was agreed, and a later price change must not silently rewrite it. This is the kind of decision service boundaries force you to make explicitly, which is an argument in their favour.

Step 3: changing configuration on a running service

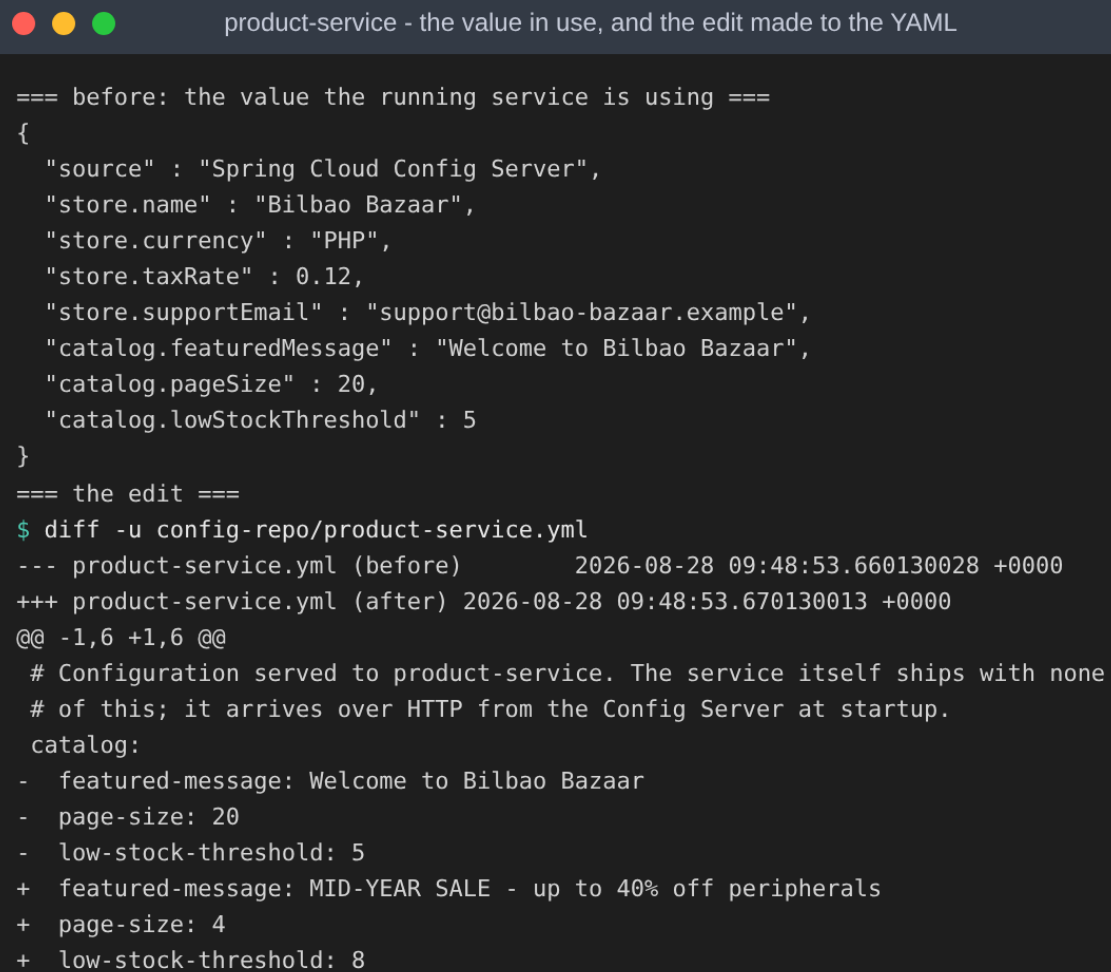
The assignment asks for configuration values to be updated and retrieved. Retrieving them is already shown. Updating them is more interesting than it sounds, because there are three separate states and it is easy to mistake one for another.



```
17 @Component
18 @RefreshScope
19 @ConfigurationProperties(prefix = "catalog")
20 public class CatalogProperties {
21
22     /** Banner text shown with the catalogue. Changed in Step 3 to prove refresh works. */
23     private String featuredMessage = "(not configured)";
24
25     /** How many products a page of the catalogue holds. */
26     private int pageSize = 10;
27
28     /** At or below this quantity a product is reported as low stock. */
29     private int lowStockThreshold = 1;
30 }
```

product-service/src/main/java/com/bilbao/ecommerce/product/CatalogProperties.java Java Spaces: 4 UTF-8

Figure 31: `@RefreshScope` is what makes an update possible without a restart. Without it these values are bound once at startup; with it, the bean is discarded and rebuilt on next use after a refresh.

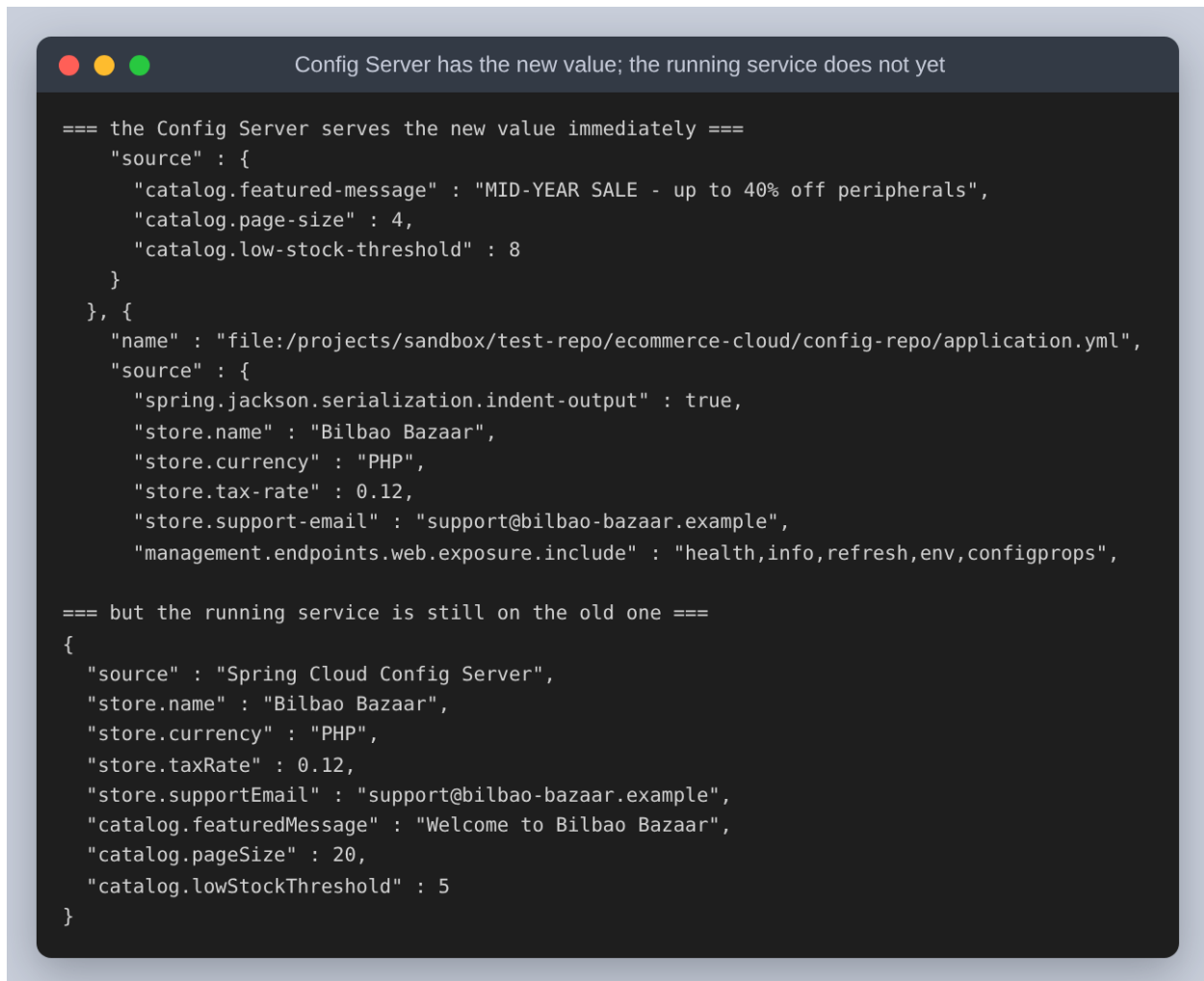


```
product-service - the value in use, and the edit made to the YAML

=== before: the value the running service is using ===
{
  "source" : "Spring Cloud Config Server",
  "store.name" : "Bilbao Bazaar",
  "store.currency" : "PHP",
  "store.taxRate" : 0.12,
  "store.supportEmail" : "support@bilbao-bazaar.example",
  "catalog.featuredMessage" : "Welcome to Bilbao Bazaar",
  "catalog.pageSize" : 20,
  "catalog.lowStockThreshold" : 5
}

=== the edit ===
$ diff -u config-repo/product-service.yml
--- product-service.yml (before)      2026-08-28 09:48:53.660130028 +0000
+++ product-service.yml (after) 2026-08-28 09:48:53.670130013 +0000
@@ -1,6 +1,6 @@
 # Configuration served to product-service. The service itself ships with none
 # of this; it arrives over HTTP from the Config Server at startup.
 catalog:
- featured-message: Welcome to Bilbao Bazaar
- page-size: 20
- low-stock-threshold: 5
+ featured-message: MID-YEAR SALE - up to 40% off peripherals
+ page-size: 4
+ low-stock-threshold: 8
```

Figure 32: The value the running service is using, then the edit: a new banner, the page size cut from 20 to 4, and the low-stock threshold raised from 5 to 8.



```
Config Server has the new value; the running service does not yet

=== the Config Server serves the new value immediately ===
"source" : {
  "catalog.featured-message" : "MID-YEAR SALE - up to 40% off peripherals",
  "catalog.page-size" : 4,
  "catalog.low-stock-threshold" : 8
}
}, {
  "name" : "file:/projects/sandbox/test-repo/ecommerce-cloud/config-repo/application.yml",
  "source" : {
    "spring.jackson.serialization.indent-output" : true,
    "store.name" : "Bilbao Bazaar",
    "store.currency" : "PHP",
    "store.tax-rate" : 0.12,
    "store.support-email" : "support@bilbao-bazaar.example",
    "management.endpoints.web.exposure.include" : "health,info,refresh,env,configprops",

=== but the running service is still on the old one ===
{
  "source" : "Spring Cloud Config Server",
  "store.name" : "Bilbao Bazaar",
  "store.currency" : "PHP",
  "store.taxRate" : 0.12,
  "store.supportEmail" : "support@bilbao-bazaar.example",
  "catalog.featuredMessage" : "Welcome to Bilbao Bazaar",
  "catalog.pageSize" : 20,
  "catalog.lowStockThreshold" : 5
}
```

Figure 33: The state that is easy to miss. The Config Server serves the new values the moment the file is saved, and the running service is still on the old ones. Checking the Config Server alone would have looked like success.

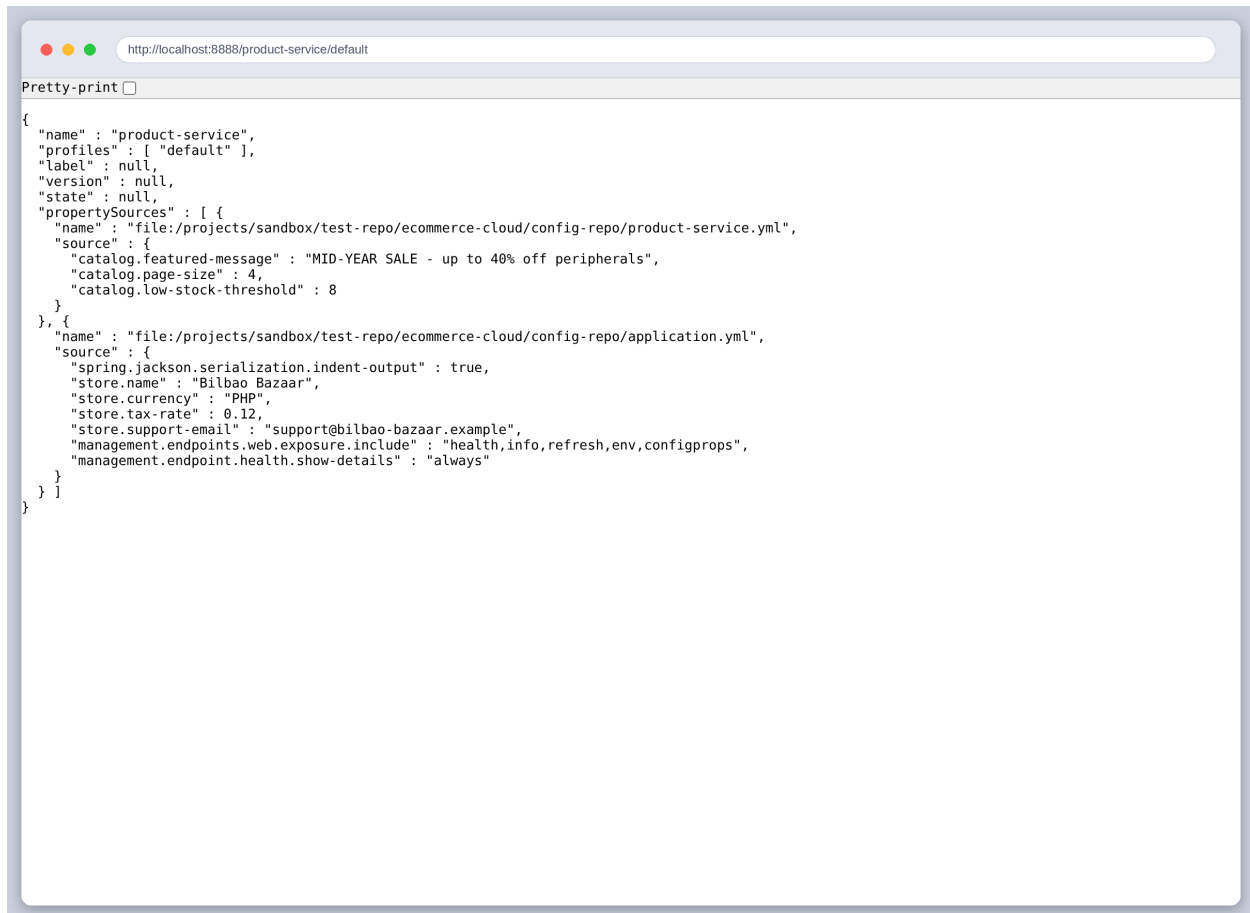
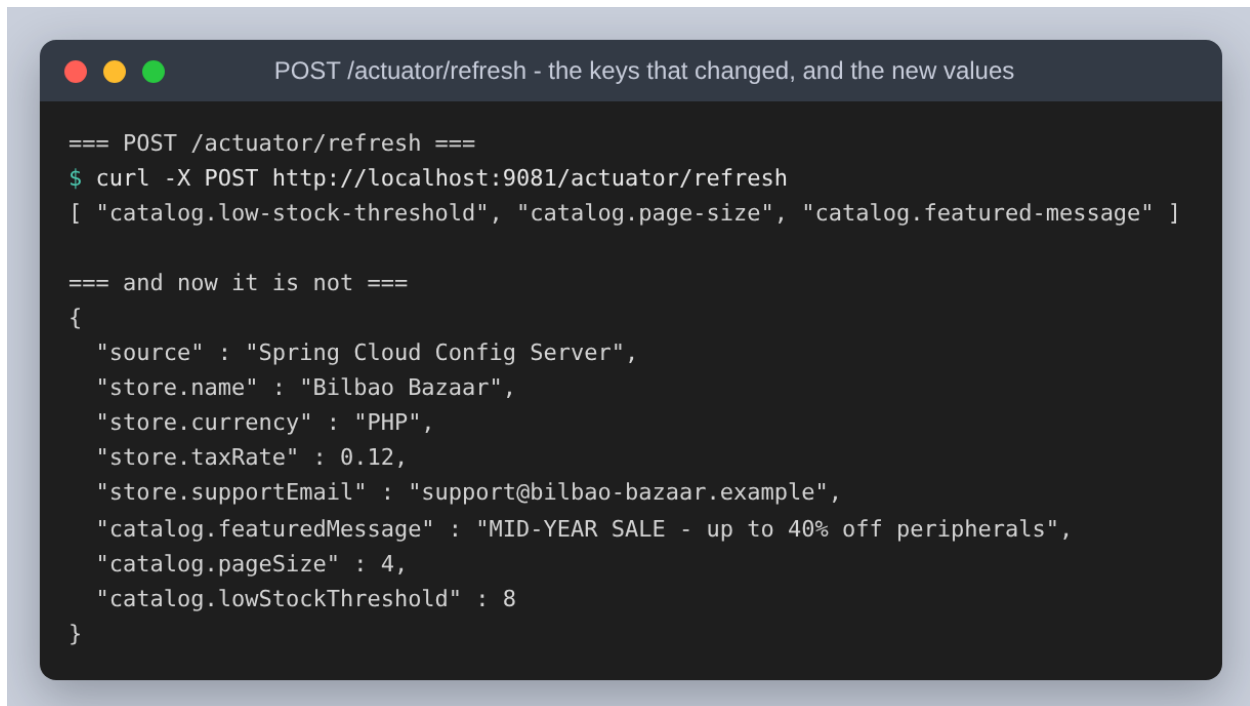


Figure 34: The Config Server in a browser after the edit, serving the new values.

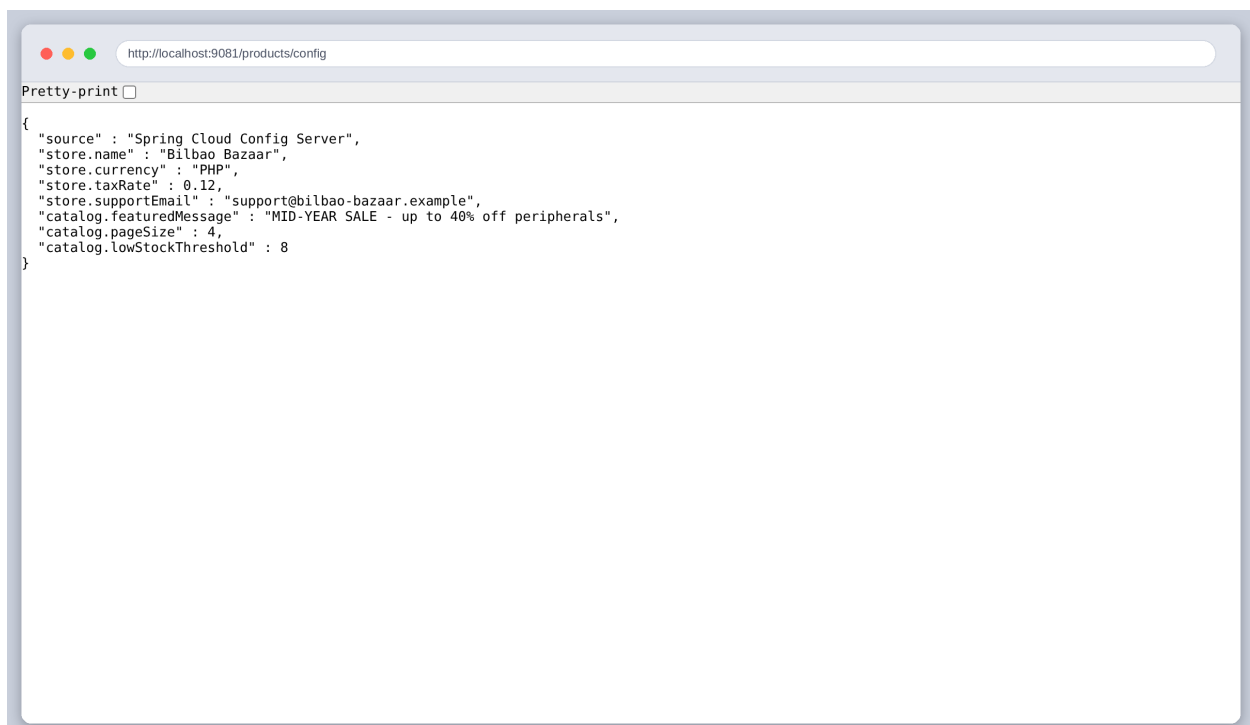


```
POST /actuator/refresh - the keys that changed, and the new values

=== POST /actuator/refresh ===
$ curl -X POST http://localhost:9081/actuator/refresh
[ "catalog.low-stock-threshold", "catalog.page-size", "catalog.featured-message" ]

=== and now it is not ===
{
  "source" : "Spring Cloud Config Server",
  "store.name" : "Bilbao Bazaar",
  "store.currency" : "PHP",
  "store.taxRate" : 0.12,
  "store.supportEmail" : "support@bilbao-bazaar.example",
  "catalog.featuredMessage" : "MID-YEAR SALE - up to 40% off peripherals",
  "catalog.pageSize" : 4,
  "catalog.lowStockThreshold" : 8
}
```

Figure 35: `POST /actuator/refresh` returns `[ "catalog.low-stock-threshold", "catalog.page-size", "catalog.featured-message" ]`, the keys it found changed, and the service now reports the new values.



```
http://localhost:9081/products/config

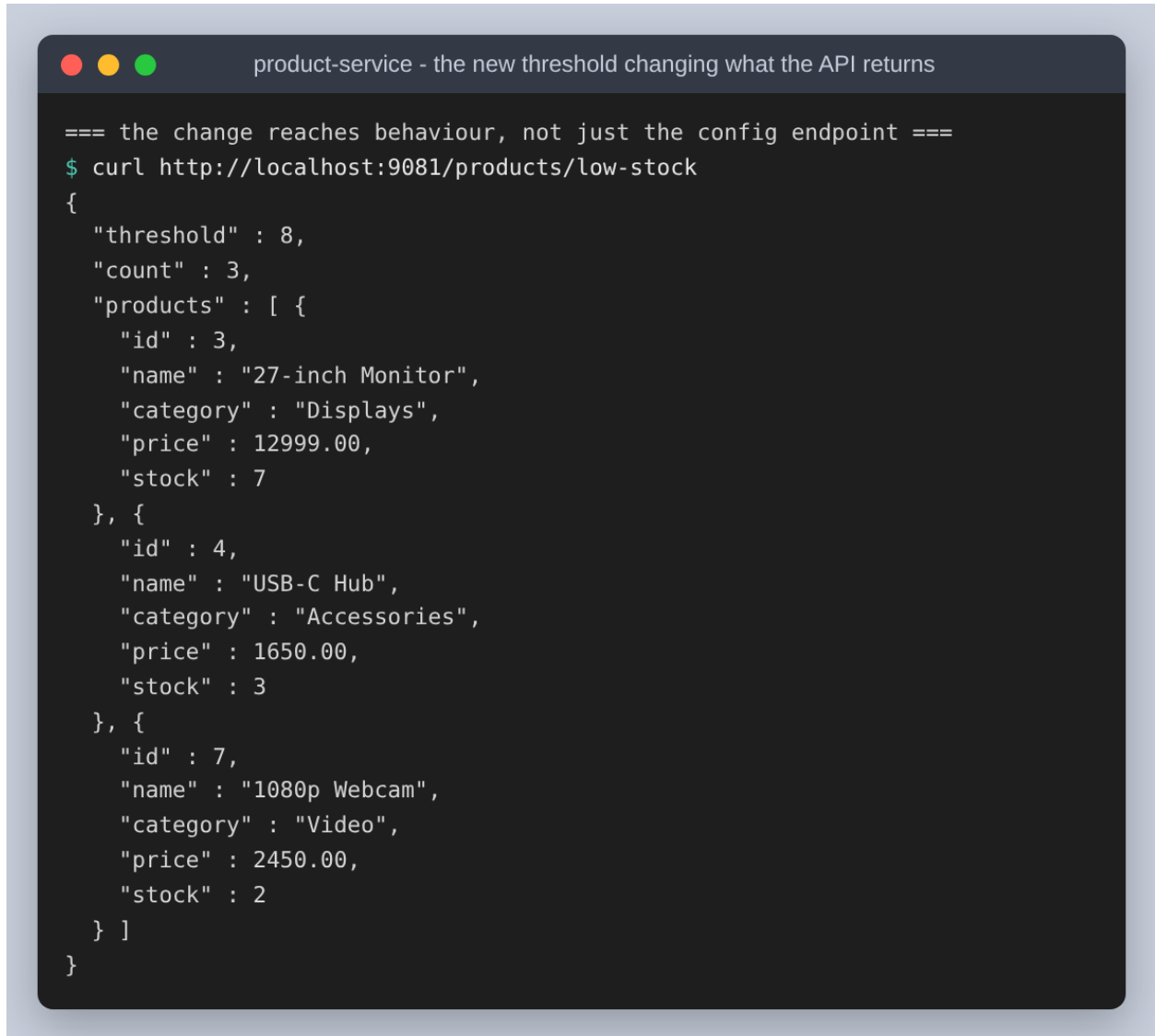
Pretty-print ☐

{
  "source" : "Spring Cloud Config Server",
  "store.name" : "Bilbao Bazaar",
  "store.currency" : "PHP",
  "store.taxRate" : 0.12,
  "store.supportEmail" : "support@bilbao-bazaar.example",
  "catalog.featuredMessage" : "MID-YEAR SALE - up to 40% off peripherals",
  "catalog.pageSize" : 4,
  "catalog.lowStockThreshold" : 8
}
```

Figure 36: The same endpoint in a browser after the refresh. Compare with the earlier screenshot of this URL.

A configuration endpoint reporting a new number is still only a report. The threshold is used to decide which products count as low stock, so raising it from 5 to 8 should change what a different

endpoint returns:

A terminal window titled "product-service - the new threshold changing what the API returns" displays the output of a curl command. The output is a JSON object with a "threshold" of 8, a "count" of 3, and an array of three products. The first product is a "27-inch Monitor" with a stock of 7. The second is a "USB-C Hub" with a stock of 3. The third is a "1080p Webcam" with a stock of 2. The terminal has a dark background with light-colored text.

```
product-service - the new threshold changing what the API returns

=== the change reaches behaviour, not just the config endpoint ===
$ curl http://localhost:9081/products/low-stock
{
  "threshold" : 8,
  "count" : 3,
  "products" : [ {
    "id" : 3,
    "name" : "27-inch Monitor",
    "category" : "Displays",
    "price" : 12999.00,
    "stock" : 7
  }, {
    "id" : 4,
    "name" : "USB-C Hub",
    "category" : "Accessories",
    "price" : 1650.00,
    "stock" : 3
  }, {
    "id" : 7,
    "name" : "1080p Webcam",
    "category" : "Video",
    "price" : 2450.00,
    "stock" : 2
  } ]
}
```

Figure 37: The new threshold changing behaviour: 3 products now count as low stock, including the monitor with 7 in stock, which did not qualify under the old threshold of 5.

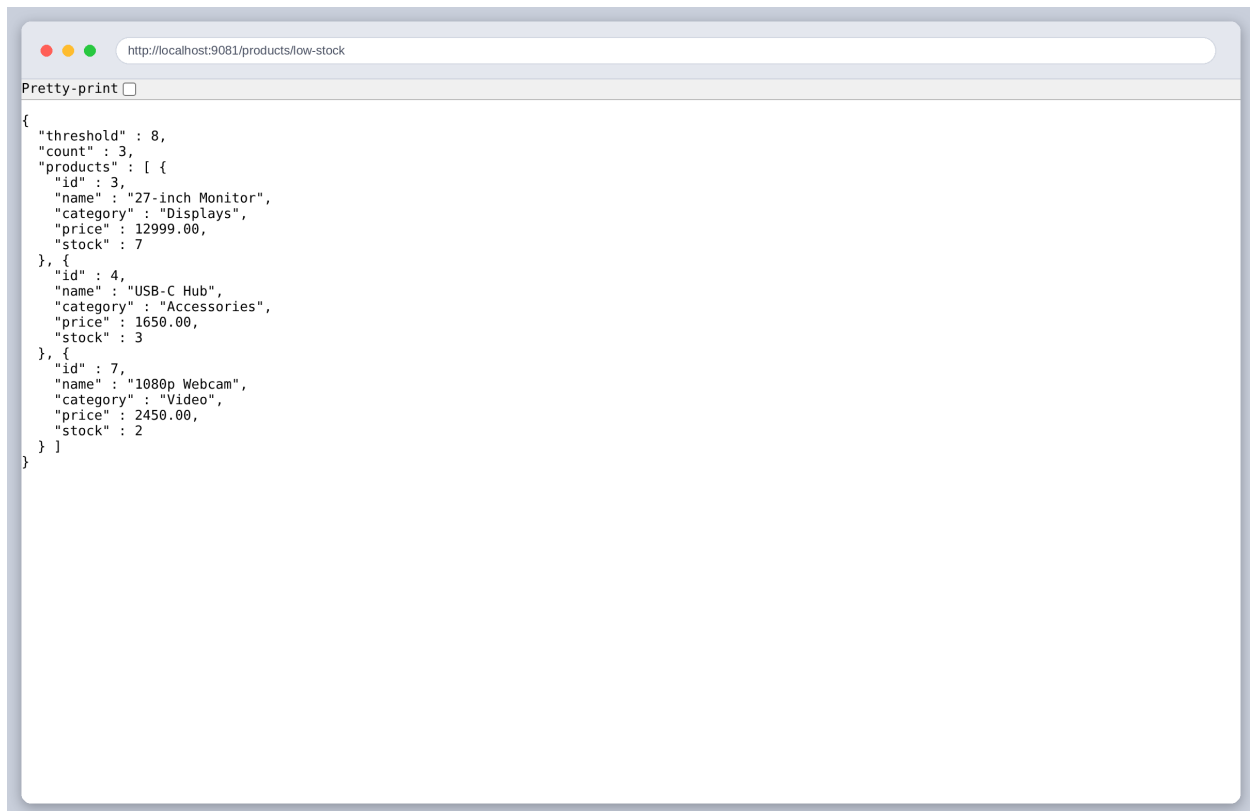
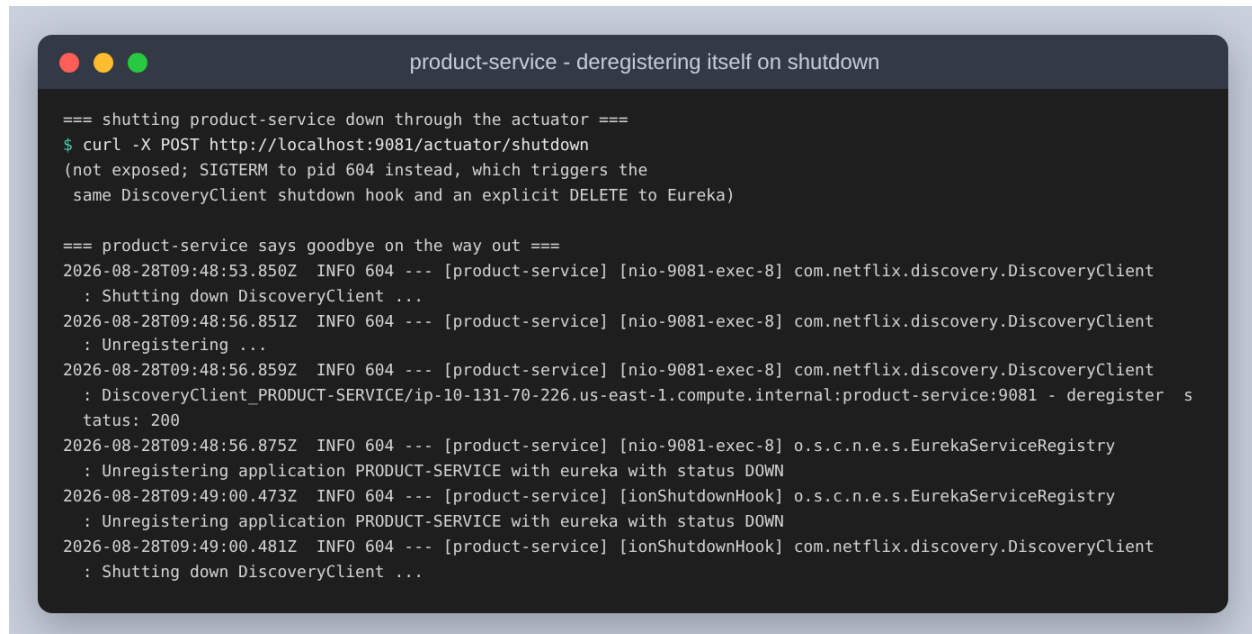


Figure 38: The same result in a browser.

Step 2 again: deregistering, and coming back somewhere else

The assignment asks for service discovery to be tested by registering and deregistering. Registering is shown above. Deregistering is where a registry earns its place, because the interesting question is what happens to the caller.

A terminal window titled "product-service - deregistering itself on shutdown" with three colored window control buttons (red, yellow, green) in the top-left corner. The terminal displays a series of log messages and a command execution. The logs show the shutdown of the DiscoveryClient, its deregistration from Eureka, and the final shutdown of the client. The command executed is a curl POST request to the actuator/shutdown endpoint, which returns a 200 status code.

```
=== shutting product-service down through the actuator ===
$ curl -X POST http://localhost:9081/actuator/shutdown
(not exposed; SIGTERM to pid 604 instead, which triggers the
 same DiscoveryClient shutdown hook and an explicit DELETE to Eureka)

=== product-service says goodbye on the way out ===
2026-08-28T09:48:53.850Z INFO 604 --- [product-service] [nio-9081-exec-8] com.netflix.discovery.DiscoveryClient
: Shutting down DiscoveryClient ...
2026-08-28T09:48:56.851Z INFO 604 --- [product-service] [nio-9081-exec-8] com.netflix.discovery.DiscoveryClient
: Unregistering ...
2026-08-28T09:48:56.859Z INFO 604 --- [product-service] [nio-9081-exec-8] com.netflix.discovery.DiscoveryClient
: DiscoveryClient_PRODUCT-SERVICE/ip-10-131-70-226.us-east-1.compute.internal:product-service:9081 - deregister s
tatus: 200
2026-08-28T09:48:56.875Z INFO 604 --- [product-service] [nio-9081-exec-8] o.s.c.n.e.s.EurekaServiceRegistry
: Unregistering application PRODUCT-SERVICE with eureka with status DOWN
2026-08-28T09:49:00.473Z INFO 604 --- [product-service] [ionShutdownHook] o.s.c.n.e.s.EurekaServiceRegistry
: Unregistering application PRODUCT-SERVICE with eureka with status DOWN
2026-08-28T09:49:00.481Z INFO 604 --- [product-service] [ionShutdownHook] com.netflix.discovery.DiscoveryClient
: Shutting down DiscoveryClient ...
```

Figure 39: The Product Service on the way out. Spring's shutdown hook runs the `DiscoveryClient` shutdown, which sends an explicit deregistration and gets 200 back. A service that is stopped cleanly does not wait to be evicted; it says it is going.

http://localhost:8761/

spring Eureka

HOME LAST 1000 SINCE STARTUP

System Status

Environment	test	Current time	2026-08-28T09:49:15 +0000
Data center	default	Uptime	00:00
		Lease expiration enabled	true
		Renews threshold	3
		Renews (last min)	0

THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.

DS Replicas

localhost

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
ORDER-SERVICE	n/a (1)	(1)	UP (1) - ip-10-131-70-226.us-east-1.compute.internal:order-service:9082

General Info

Name	Value
total-avail-memory	160mb
num-of-cpus	8
current-memory-usage	96mb (60%)
server-uptime	00:00
registered-replicas	http://localhost:8761/eureka/
unavailable-replicas	http://localhost:8761/eureka/ ,
available-replicas	

Instance Info

Name	Value
ipAddr	169.254.255.194
status	UP

Figure 40: The dashboard with only ORDER-SERVICE left. The assignment's deregistration test, in the browser.

```
Eureka - the registry with product-service gone

=== the registry no longer lists PRODUCT-SERVICE ===
$ curl -H 'Accept: application/json' http://localhost:8761/eureka/apps
{"applications":{"versions__delta":"1","apps__hashcode":"UP_1_","application":[{"name":"ORDER-SERVICE","instance":{"instanceId":"ip-10-131-70-226.us-east-1.compute.internal:order-service:9082","hostName":"169.254.255.194","app":"ORDER-SERVICE","ipAddr":"169.254.255.194","status":"UP","overriddenStatus":"UNKNOWN","port":{"$":9082,"@enabled":"true"},"securePort":{"$":443,"@enabled":"false"},"countryId":1,"dataCenterInfo":{"@class":"com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo","name":"MyOwn"},"leaseInfo":{"renewalIntervalInSecs":5,"durationInSecs":10,"registrationTimestamp":1787910511854,"lastRenewalTimestamp":1787910551862,"evictionTimestamp":0,"serviceUpTimestamp":1787910511341},"metadata":{"management.port":"9082"},"homePageUrl":"http://169.254.255.194:9082/","statusPageUrl":"http://169.254.255.194:9082/actuator/info","healthCheckUrl":"http://169.254.255.194:9082/actuator/health","vipAddress":"order-service","secureVipAddress":"order-service","isCoordinatingDiscoveryServer":"false","lastUpdatedTimestamp":"1787910511854","lastDirtyTimestamp":"1787910511310","actionType":"ADD ED"}]}]}

=== order-service sees an empty instance list ===
{
  "servicesRegisteredInEureka" : [ "order-service" ],
  "productServiceInstances" : [ ],
  "configurationFromConfigServer" : {
    "store.name" : "Bilbao Bazaar",
    "store.currency" : "PHP",
    "store.taxRate" : 0.12,
    "orders.maxItemsPerOrder" : 10,
    "orders.freeShippingThreshold" : 2000,
    "orders.shippingFee" : 150
  }
}
```

Figure 41: The same absence over the REST API.

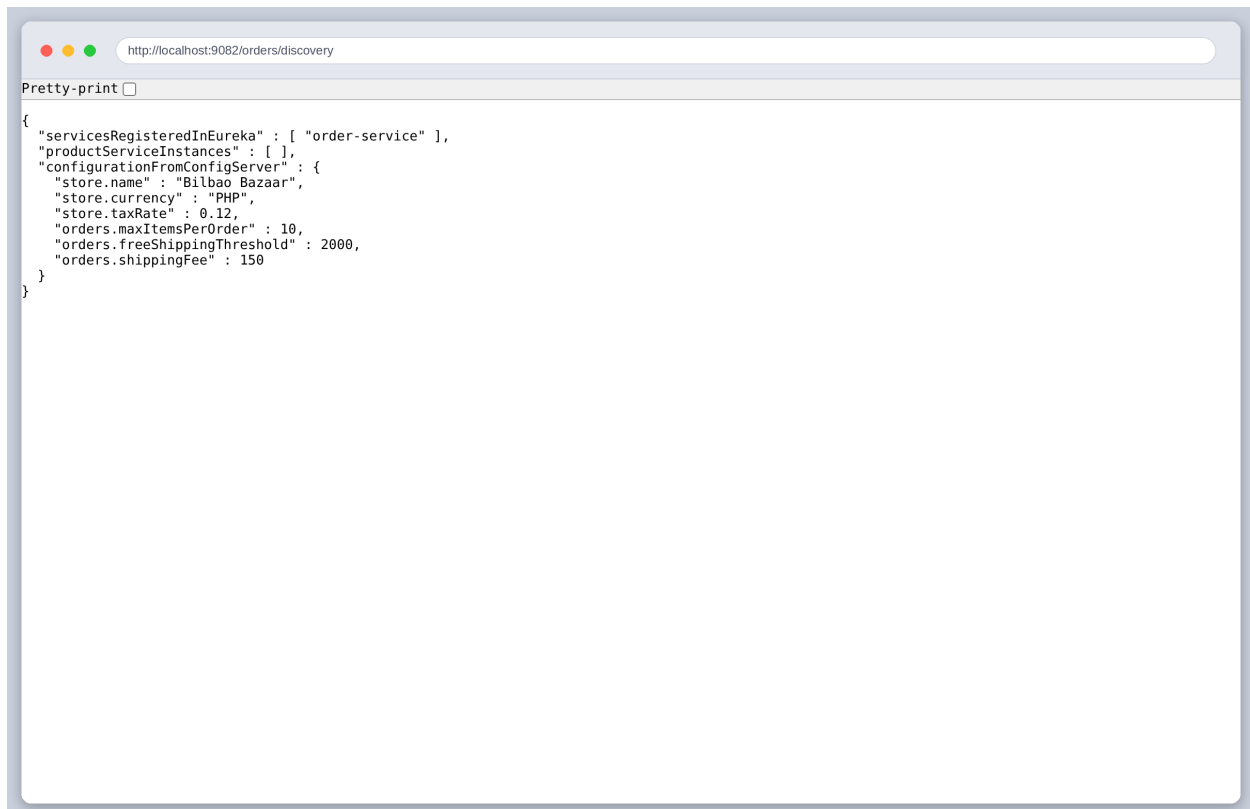


Figure 42: The Order Service now sees an empty instance list for product-service, while its own configuration is untouched. Discovery and configuration fail independently, which is worth knowing when something breaks.

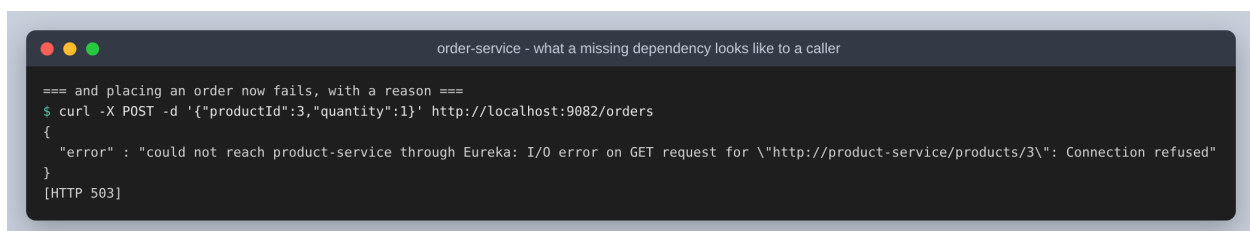
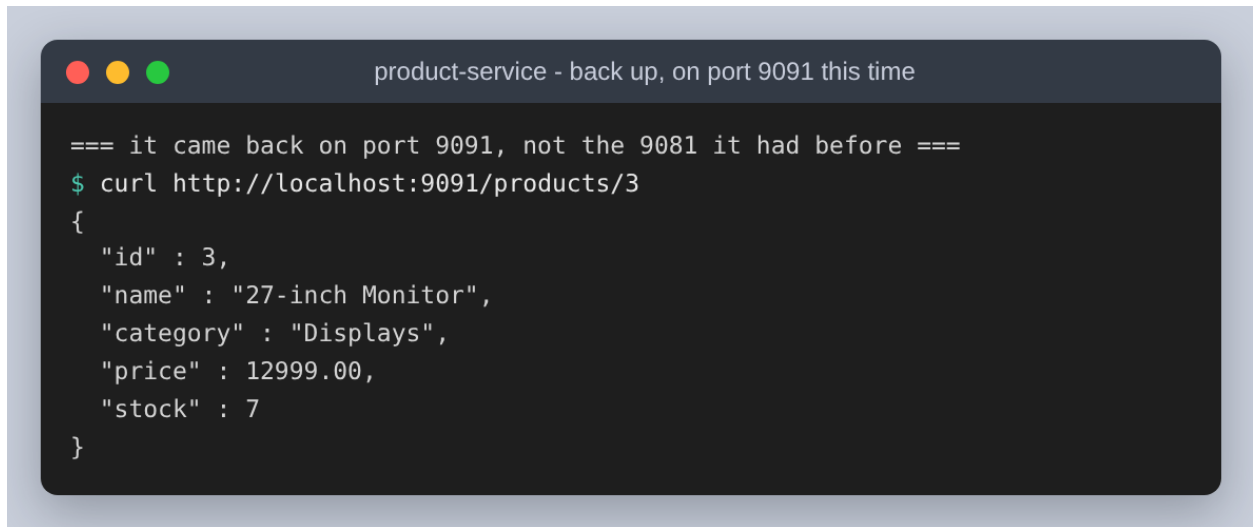


Figure 43: Placing an order now fails with 503 and a readable reason rather than a stack trace. A missing dependency is an operational state, not a bug in the Order Service, and orders already placed are unaffected.

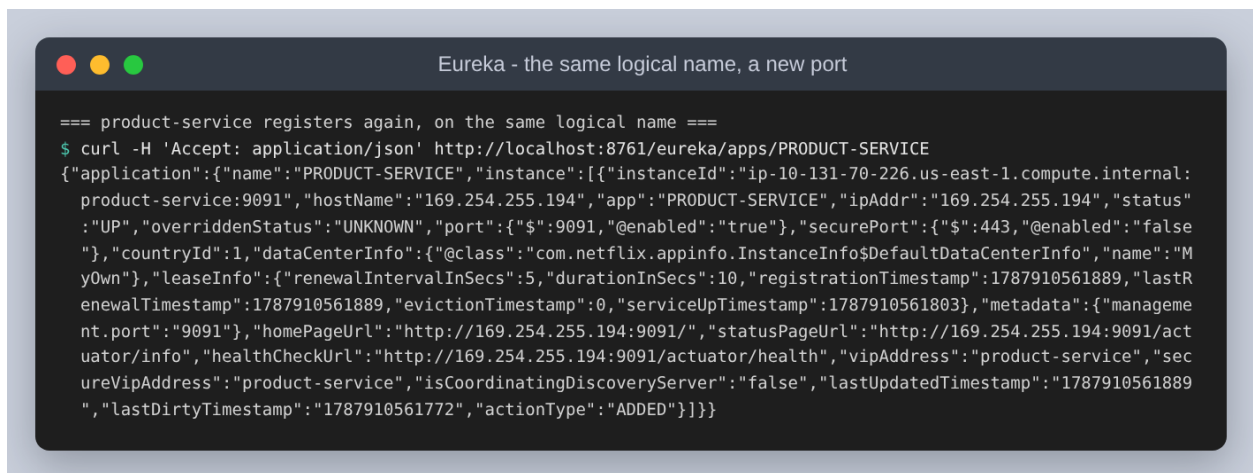
Then the Product Service is started again, deliberately on port 9091 rather than the 9081 it had before. Nothing in the Order Service is changed, and it is not restarted.



```
product-service - back up, on port 9091 this time

=== it came back on port 9091, not the 9081 it had before ===
$ curl http://localhost:9091/products/3
{
  "id" : 3,
  "name" : "27-inch Monitor",
  "category" : "Displays",
  "price" : 12999.00,
  "stock" : 7
}
```

Figure 44: The Product Service answering on 9091.



```
Eureka - the same logical name, a new port

=== product-service registers again, on the same logical name ===
$ curl -H 'Accept: application/json' http://localhost:8761/eureka/apps/PRODUCT-SERVICE
{"application":{"name":"PRODUCT-SERVICE","instance":[{"instanceId":"ip-10-131-70-226.us-east-1.compute.internal:product-service:9091","hostName":"169.254.255.194","app":"PRODUCT-SERVICE","ipAddr":"169.254.255.194","status":"UP","overriddenStatus":"UNKNOWN","port":{"$":9091,"@enabled":"true"},"securePort":{"$":443,"@enabled":"false"},"countryId":1,"dataCenterInfo":{"@class":"com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo","name":"MyOwn"},"leaseInfo":{"renewalIntervalInSecs":5,"durationInSecs":10,"registrationTimestamp":1787910561889,"lastRenewalTimestamp":1787910561889,"evictionTimestamp":0,"serviceUpTimestamp":1787910561803},"metadata":{"management.port":"9091"},"homePageUrl":"http://169.254.255.194:9091/","statusPageUrl":"http://169.254.255.194:9091/actuator/info","healthCheckUrl":"http://169.254.255.194:9091/actuator/health","vipAddress":"product-service","secureVipAddress":"product-service","isCoordinatingDiscoveryServer":false,"lastUpdatedTimestamp":1787910561889,"lastDirtyTimestamp":1787910561772,"actionType":"ADDED"}]}}
```

Figure 45: It registers under the same logical name, PRODUCT-SERVICE, with the new port in the lease.

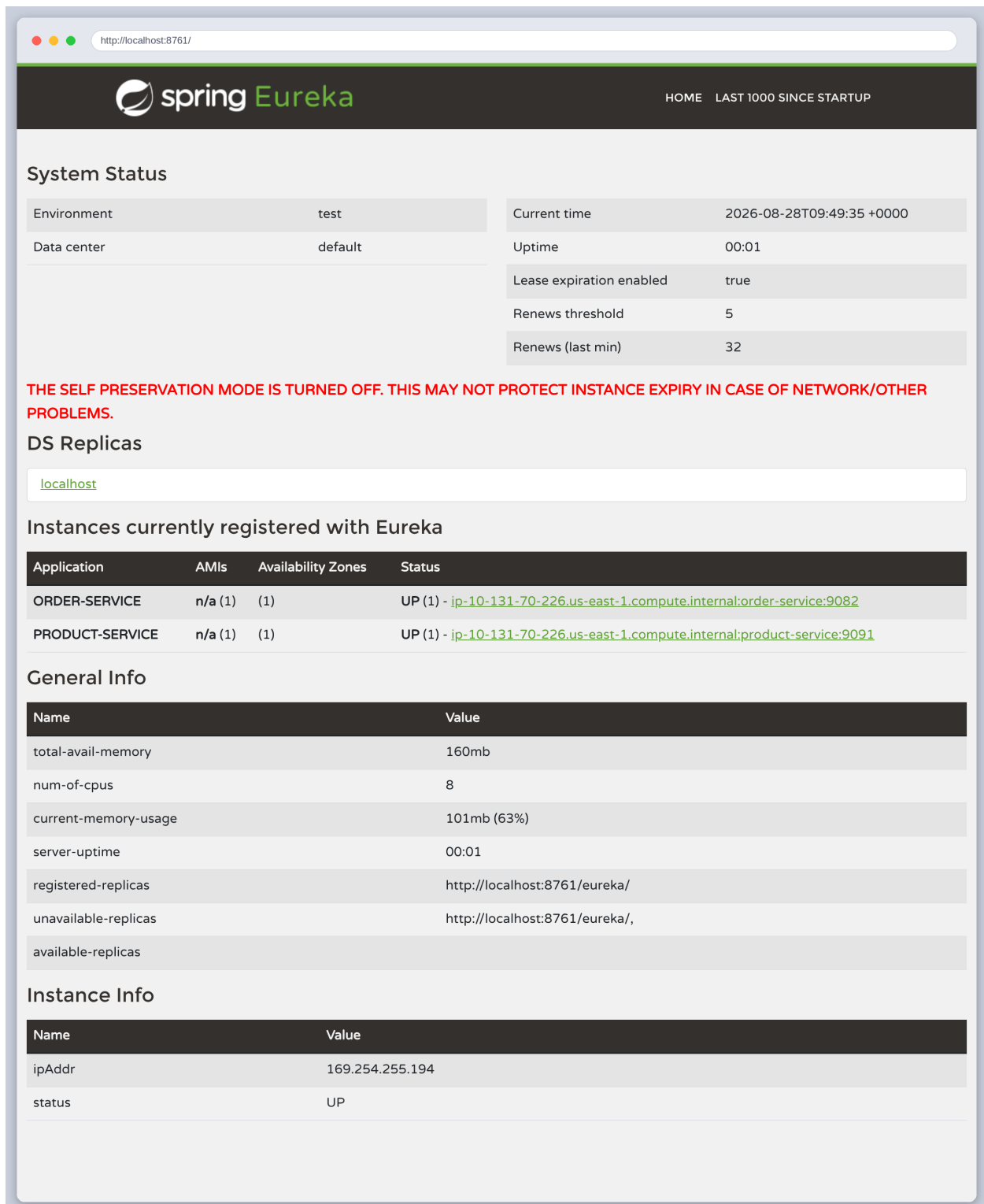


Figure 46: The dashboard: both services UP again, product-service now on 9091.

```
order-service - finds it at the new port and prices an order, with no restart

=== order-service finds it again, at the new port, with no restart ===
=== and no configuration change of its own ===
{
  "servicesRegisteredInEureka" : [ "order-service", "product-service" ],
  "productServiceInstances" : [ {
    "instanceId" : "ip-10-131-70-226.us-east-1.compute.internal:product-service:9091",
    "serviceId" : "PRODUCT-SERVICE",
    "host" : "169.254.255.194",
    "port" : 9091,
    "uri" : "http://169.254.255.194:9091"
  } ],
  "configurationFromConfigServer" : {
    "store.name" : "Bilbao Bazaar",
    "store.currency" : "PHP",
    "store.taxRate" : 0.12,
    "orders.maxItemsPerOrder" : 10,
    "orders.freeShippingThreshold" : 2000,
    "orders.shippingFee" : 150
  }
}
=== and orders work again ===
{
  "id" : "c2ff813c",
  "productId" : 6,
  "productName" : "Noise-cancelling Headset",
  "quantity" : 2,
  "unitPrice" : 5799.0,
  "subtotal" : 11598.00,
  "tax" : 1391.76,
  "shipping" : 0.00,
  "total" : 12989.76,
  "currency" : "PHP",
  "servedByProductInstance" : "169.254.255.194:9091",
  "placedAt" : "2026-08-28T09:49:34.532114269Z"
}
[HTTP 201]
```

Figure 47: The payoff. The Order Service finds it at the new port and prices an order, total 12989.76, served by 169.254.255.194:9091 - with no restart and no configuration change of its own.

This is the part of the assignment that is hard to fake and easy to underrate. Had the Order Service held the Product Service's address in a config file, this restart would have broken it, and fixing it would have meant editing configuration and refreshing. Because the address is resolved from the registry on every call, the move cost nothing. That is the entire practical argument for service discovery, and it took a port change to actually demonstrate it.

How it was implemented, in order

- One aggregator pom with the Spring Cloud BOM imported once, so the four modules cannot disagree on versions.
- Config Server: spring-cloud-config-server and `@EnableConfigServer`, pointed at config-repo/ with the native profile. Shared values in application.yml, per-service values in a file named after the service's spring.application.name.
- Eureka server: spring-cloud-starter-netflix-eureka-server and `@EnableEurekaServer`, with register-with-eureka and fetch-registry off because it is the registry, and self preservation off so eviction can be observed.
- Both business services: the Eureka client starter to register, spring-cloud-starter-config plus spring.config.import to be configured at startup, and actuator to expose /actuator/refresh.
- `@RefreshScope` on the `@ConfigurationProperties` beans, which is what lets POST /actuator/refresh rebind them without a restart.
- A `@LoadBalanced RestTemplate` in the Order Service, so `http://product-service/...` resolves through the registry. This is the only place discovery is actually consumed rather than merely configured.

The two pieces of code that matter

Everything else is annotations and YAML. These two are the load-bearing parts, quoted from the source.

```
@Bean
@LoadBalanced
public RestTemplate loadBalancedRestTemplate() {
```

And the call that uses it, where the logical name appears instead of an address:

```
public ProductView fetch(long productId) {
    Map body = loadBalanced.getForObject(
        "http://" + serviceName + "/products/{id}", Map.class, productId);
    if (body == null) {
        throw new ProductLookupException(
            "product-service returned an empty body for product " + productId);
    }
}
```

Reproducing it

```
cd ecommerce-cloud
./run.sh                # builds, starts all four, exercises everything
python3 scripts/make_figures.py # results/ and screenshots/ into figures/
python3 build.py         # figures/ into the .docx and .pdf
```

run.sh does the whole thing in one invocation, because the four services are its child processes and splitting the steps across separate shells would lose them in between. It starts them in

dependency order, waits on each health endpoint, exercises both mechanisms, drives a real browser for the screenshots, and shuts everything down and restores the edited YAML on the way out.

Conclusion and takeaways

Both mechanisms did what they claim, and building them was less work than expected: the Config Server and the Eureka server are one annotation each. What took the actual thinking was working out how to show they were working rather than merely running.

- A registry is only doing something if the caller uses it. Registering both services and screenshotting the dashboard demonstrates almost nothing on its own; the same dashboard would look identical if the Order Service had the Product Service's port hard-coded. Restarting the Product Service on a different port is what turns the claim into evidence.
- The failing case is the clearest proof. `UnknownHostException` on a plain `RestTemplate` shows that the logical name is genuinely unresolvable, and therefore that the working call is doing a registry lookup. A test that only ever passes cannot tell you which mechanism made it pass.
- There are three states in a configuration change, not two: the file, the Config Server's view of it, and the running service's view of it. The Config Server picked up the edit immediately while the service was still serving old values. Checking the server alone would have looked like success and been wrong.
- Refreshing a value is not the same as changing behaviour. The config endpoint reported the new threshold, and it was worth checking a second endpoint that uses it to confirm the new value was actually being applied.
- Startup order is a real dependency. The business services fetch configuration during startup, so the Config Server has to be up first. Using `optional: in spring.config.import` means they still start without it, which is a better failure than refusing to boot, but they start unconfigured.
- Ordinary defaults get in the way of demonstrating things. Eureka's self preservation and its 30-second lease intervals are correct for production and wrong for an assignment that needs to show a service disappearing within a screenshot's patience.